

Mock Exam 3 — Claude Certified Developer – Foundations (CCDV-F)

50 questions · 115 minutes · single best answer. Design and scoring rules: blueprint.md.

Take this cold — no notes, no repo, one pass, timed. Write your answers down, then score against Section C.

Section A — Exam Conditions

Property	This exam
Items	50
Time	115 minutes
Pace	~2.3 minutes per item
Borderline	72% raw (36/50)
Comfortable	80% raw (40/50)
Materials	Closed book — no notes, no repo, no docs, no model
Format	Single best answer, four options

Sit this one after you've cleared your top three weak areas. Its job is to confirm the reinforcement actually worked, so pay more attention to the per-domain breakdown than the total: a headline score that improved while a weak domain stayed flat means the gain came from somewhere else and the gap is still there.

Domain	Questions	Question numbers
Applications and Integration	17	1, 4, 6, 9, 11, 14, 16, 19, 22, 25, 27, 30, 33, 36, 40, 44, 47
Model Selection and Optimization	8	3, 8, 13, 20, 29, 34, 39, 48
Agents and Workflows	8	2, 12, 18, 28, 32, 38, 43, 50
Prompt and Context Engineering	5	5, 15, 24, 35, 45
Tools and MCP	5	7, 17, 31, 41, 49
Security and Safety	4	10, 21, 37, 46
Claude Code	2	23, 42
Eval, Testing, and Debugging	1	26
Total	50	

Section B — Questions

Q1

A dental practice network wants Claude to draft insurance narratives for completed procedures. At kickoff the lead engineer's first deliverable is a deployment diagram choosing between a streaming endpoint and a nightly batch pipeline, while the behaviors the feature must produce — which chart fields a narrative cites, whether a hygienist approves before submission — are still unwritten. What does the solution-architecture guidance say about that **order** of work?

- **A.** A solution architecture maps functional requirements onto infrastructure choices, so each behavior is pinned first and then forces its infrastructure choice; picking the infrastructure pattern before the requirement is pinned is the root cause behind most "why is this slow or expensive" complaints these systems generate.
- **B.** The order is fine, because the four infrastructure dimensions — latency, scale, residency, identity — are read directly off the business problem as stated and can therefore be settled before any functional behavior is written down.
- **C.** The order is fine as long as the diagram is revisited at the design → build gate, which is the point at which functional requirements are derived from the platform that was chosen.
- **D.** Neither ordering matters much here, because infrastructure choice follows from the model tier, and the tier is settled empirically against an eval once the feature has been built.

Q2

A port authority's compliance team wants Claude to flag cargo manifests that may hide a hazardous-goods misdeclaration. There is one judgment to reach per manifest, the cost of missing one is high, and the team wants several independent reads of the **same** manifest combined into a confidence signal rather than a faster pass over separate parts of it. Which form of parallelization is that, and what does it buy?

- **A.** Sectioning — the manifest is split into independent parts examined simultaneously, so the review finishes in the time of the slowest part rather than the sum of all of them.
- **B.** Voting — multiple attempts at the same task run simultaneously to get diverse perspectives on one judgment; sectioning is the other form of parallelization and is what you reach for when independent subtasks make it a speed problem.
- **C.** Evaluator-optimizer — one call produces the flag and a second evaluates it and feeds back corrections, iterating until the confidence criteria are met.
- **D.** Orchestrator-workers — a lead call dynamically decomposes the manifest review into subtasks it cannot predict in advance and dispatches each one to a worker.

Q3

A textile mill's shift-log summarizer truncates each log before sending it, using a hardcoded estimate of four characters per token that was validated when the service launched on a pre-4.7 model. After the team migrates the service to Fable 5, logs the truncation step believes are comfortably inside the input allowance keep arriving over budget. What explains it?

- **A.** Fable 5 has a smaller context window than the generation the estimate was validated against, so the same token count no longer fits; raising the truncation allowance restores the old behavior.
- **B.** The estimate is sound, but tool definitions and tool results now draw on the same window as the prompt, which they did not on the earlier generation.
- **C.** The characters-per-token ratio is a property of the model's **tokenizer** and changes across model generations — Fable 5 moved to the tokenizer introduced with Opus 4.7, which produces roughly 30% more tokens for the same text — so a hardcoded constant silently understates the real count and has to be confirmed against current tokenizer behavior at build time.
- **D.** Characters per token is fixed by the API and identical across Claude models, so the discrepancy has to be an encoding problem in how the logs are read, since multi-byte characters count as more than one character.

Q4

A concert ticketing platform is planning a refund-triage feature. The project plan schedules construction of the eval suite for after the agent is written, as the first activity of the **test** phase, on the reasoning that an eval is a test. Where does the lifecycle guidance place that work, and why?

- **A.** In the test phase, alongside the unit, integration, and end-to-end checks, since all four levels of testing belong to the same phase and share the same fixtures.
- **B.** In the operate phase, because the only cases worth grading are the failures real traffic produces, and cases written earlier grade behavior nobody has observed yet.
- **C.** In the requirements phase, where the eval suite is written as the requirements record itself and replaces the separate functional/infrastructure write-up.
- **D.** In the build phase, alongside the agent, tools, and prompts: the suite is what gates the deploy phase against a pinned baseline score and what makes iterate → operate tractable once the system is live, so building it as an afterthought leaves both of those phases with nothing but manual spot-checking.

Q5

A municipal transit agency's service-alert assistant has a carefully tuned system prompt, but nobody has ever measured how much of each call is taken by injected timetable extracts, tool definitions, and thirty turns of history. A new engineer asks how **context engineering** differs from the prompt engineering the team has been doing. Which description is right?

- **A.** Prompt engineering is about what you say; context engineering is the superset — curating the full set of tokens available at each inference call, including the prompt, conversation history, tool definitions, tool outputs, and injected documents, because context is a finite shared resource.
- **B.** They are the same discipline under two names; "context engineering" is the current term for writing sharper system prompts and better few-shot examples.
- **C.** Context engineering is a subset of prompt engineering covering the system prompt specifically, since that is the only part of a request that persists unchanged across turns.
- **D.** Prompt engineering covers everything your code sends to the model, while context engineering is a separate runtime concern owned by the SDK's automatic compaction rather than by your application.

Q6

A wine importer's duty-calculation assistant receives a turn containing two `tool_use` blocks, `lookup_tariff_code` and `get_exchange_rate`. The code appends that assistant turn to history, then sends a user turn carrying a `tool_result` for the tariff lookup **only**, intending to return the exchange rate on the next turn once a slow upstream service answers. Requests now fail validation before generation starts. Which rule is being violated?

- **A.** More than one `tool_use` block per assistant turn is rejected, so the two calls should have been issued in separate turns; returning results one at a time is the right shape but the request that produced two calls was already invalid.
- **B.** Each `tool_use` block must be answered by a matching `tool_result` in the **immediately following** user turn, matched by `tool_use_id`; when one turn issues parallel calls, their results have to return together, and a missing or mismatched ID fails request validation with no prompt-level recovery available.
- **C.** Splitting results across turns is legal as long as each carries the correct `tool_use_id`; the validation failure comes from the assistant turn's `text` block being dropped when that turn was appended to history.
- **D.** The pairing is a model-behavior expectation rather than a structural one, so a system-prompt instruction telling Claude that one result may arrive a turn late is what resolves it.

Q7

A kitchen-appliance maker's warranty agent exposes a `register_appliance` tool whose input schema marks all seven fields `required`, including `dealer_id` and `installation_date` — values many customers simply never provide. Reviewing a week of calls, the team finds Claude populating those two fields with plausible but **invented** values rather than leaving them out. What does the schema guidance say to do?

- **A.** Set `strict: true` on the tool definition so arguments are validated against the schema before the function runs, which stops an unsupported value from reaching the tool.
- **B.** Add a sentence to the tool description instructing Claude not to guess missing fields, since the description is what Claude reads when constructing a call.
- **C.** Mark a field `required` only when the call is meaningless without it — marking everything required forces Claude to fabricate values it has no basis for, so fields with sensible defaults, or where absence itself carries meaning, belong as optional.
- **D.** Split the tool into two narrower tools, one for complete registrations and one for partial ones, so each schema requires only the fields that call actually receives.

Q8

A credit bureau's dispute-summary service runs on Haiku 4.5 with `temperature: 0`, and its regression tests assert the exact summary text for each fixture. The tests pass on the developer's machine and fail intermittently in CI on unchanged inputs. A developer argues that temperature 0 makes generation **deterministic**, so something must be mutating the prompt. What is the accurate reading?

- **A.** Determinism does hold at `temperature: 0` for a non-streamed call; a streamed response is reassembled from deltas and can differ, so asserting against a synchronous call stabilizes the tests.
- **B.** `temperature: 0` is deterministic per pinned model snapshot, so intermittent variation points at CI resolving a moving alias where the local run uses the pinned full model ID.
- **C.** Setting `temperature` at all returns a 400 on current models, so the parameter never took effect and the default applies; removing it and steering through prompting restores repeatability.
- **D.** Each token is sampled from a probability distribution, so `temperature: 0` makes output more repeatable but does **not** guarantee identical output across calls — exact-text assertions are inherently flaky. Assert on the property that must hold (a required field is present, a value is in range, the output parses), and use a model-graded eval when meaning is what needs judging.

Q9

A school district's accessibility team sends scanned permission slips to Claude as image content blocks with `source.type: "url"`, pointing at their document store's pre-signed links, which expire 60 seconds after issue. Most requests succeed, but a growing share fail, and the failures cluster on slips that sat behind a slow validation step before the call went out. What is the correct account of the tradeoff they took on?

- **A.** A `url` source sends no image payload with the request, but it makes you dependent on the URL being stable, public, and reachable **at the moment Claude fetches it** — which a 60-second signed link sitting in a queue is not. Anything behind auth or on a short-lived signed URL belongs in a `base64` block, or in the Files API when the same asset is reused across requests.
- **B.** URL-sourced images are fetched during request validation rather than at generation time, so raising the client's timeout so validation finishes inside the signing window is the fix.
- **C.** A `url` source is re-fetched on every turn of a conversation, so expiry only bites in multi-turn use; a single-turn call is unaffected, which means the failures come from the validation queue rather than the image source.
- **D.** The `url` source type has partial support outside the first-party API, which is what the intermittent failures reflect; moving the slips to the Files API and referencing them by `file_id` resolves it.

Q10

A forestry contractor's harvest-planning agent reads landowner-supplied PDFs and can file cutting permits with the county. Their security write-up has a single heading, "prompt injection," under which they note that Claude is trained to refuse harmful requests. A reviewer asks them to separate the **two** threats they have merged into one. What is the distinction?

- **A.** They are two names for one attack: a jailbreak is what a successful prompt injection is called once the model complies, so one input-validation layer addresses both.
- **B.** A jailbreak targets the model's own safety constraints, while prompt injection hijacks your application's instructions — different targets, but the same layered defense shape: validate and constrain what reaches the model, *and* limit what the model is allowed to do as a result.
- **C.** Jailbreaks arrive through content the agent retrieves and prompt injection arrives in the user's own prompt, so the split is by delivery route and validating the user turn closes the more dangerous half.
- **D.** Prompt injection is a training-time concern Anthropic owns through model training and classifiers, so jailbreaks are the only one an application defends against, via least privilege on the agent's identity.

Q11

A dairy cooperative's tanker-routing assistant streams responses that can include tool calls. During a network blip the connection drops after several `input_json_delta` events but before `message_stop`. The handler catches the disconnect, JSON-parses whatever had accumulated, and dispatches the resulting `reroute_tanker` call. What should it do instead?

- **A.** Reconnect and resume the same stream from the last event it received, since everything accumulated up to the break is valid and re-requesting would pay for the same input tokens twice.
- **B.** Dispatch the call as it stands but set `is_error: true` on the resulting `tool_result`, so Claude sees the partial input and can decide whether to reissue the call itself.
- **C.** Treat a stream that breaks mid-response as a **transient** failure and retry the whole request: a `tool_use` block isn't safe to act on until the stream has closed and its full `input_json` has been reassembled, so a partial accumulation must never be passed downstream as though it were complete.
- **D.** Classify the disconnect as terminal — the request already produced output, so a retry double-charges — and surface it to the dispatcher for a manual routing decision.

Q12

A translation agency's inbound work varies wildly: one job is a single certified birth certificate, the next is a 40-file tender bundle mixing contracts, drawings, and a client glossary. How a bundle should be split cannot be known until it has been inspected, but once a piece is identified it is a bounded, well-specified translate-then-review task. Which pattern fits this shape?

- **A.** Prompt chaining — a fixed sequence of calls in which each step processes the previous step's output, which improves accuracy through focused attention per step.
- **B.** A single agent with a broad toolset, since the variability sits in the input rather than in the steps and an agent can direct its own path through it.
- **C.** Routing — classify each inbound bundle and hand it to a specialized handler with its own prompt, tools, and model per category.
- **D.** Orchestrator-workers — a central LLM **dynamically** decomposes the job into subtasks it could not predict in advance and delegates each to a worker; the decomposition is agent-like while each worker's task stays scoped and bounded, which is what makes it the bridge between a pure workflow and a pure agent.

Q13

A public aquarium's exhibit-signage generator is being ported from Python to TypeScript. The Python version uses `AsyncAnthropic` to keep the request thread free while it fans one call out per tank. A developer cannot find the equivalent async client class in the TypeScript SDK and files a bug against the SDK. What is the correct response?

- **A.** There is no separate async client class in the TypeScript SDK — its standard client is already Promise-based, so you simply `await` the calls; and in either language async buys concurrency, not lower per-request latency or lower cost.
- **B.** The TypeScript SDK is synchronous by design, so concurrency there has to come from worker threads or from moving the fan-out to the Message Batches API.
- **C.** `AsyncAnthropic` maps to constructing the TypeScript client in streaming mode, since streaming over server-sent events is how that SDK yields the thread between chunks.
- **D.** The async client is what unlocks lower per-call latency, so the ported service will be measurably slower per sign until an equivalent class ships.

Q14

A bicycle courier company's dispatch app asks Claude to pick the best rider for each incoming job while the customer watches a spinner in the booking screen. Looking at the monthly bill, an engineer proposes moving these calls to the Message Batches API for its lower per-token rate. What is wrong with that?

- **A.** Nothing, as long as each request carries a `custom_id`, since that is what lets the app match a returned result back to the job that requested it.
- **B.** Realtime — synchronous or streaming — is the right choice whenever a user or a downstream process is **waiting** on the result now; the Message Batches API is for latency-tolerant work, and its lower per-token cost is bought with up to 24 hours of turnaround that a booking spinner cannot absorb.
- **C.** Batching only lowers the per-token rate above 100,000 requests per submission, so a per-job call would pay the standard rate anyway and gain nothing.
- **D.** The saving is real, but the app would have to poll for completion, and the polling requests cost more in volume than the per-token discount returns.

Q15

A cement plant's reliability engineer runs one long session that has to hold two things at once: a wide exploratory sweep through years of kiln vibration logs — dozens of large tool reads, most of which turn out irrelevant, and many of which must stay side by side to compare across years — and a remediation memo being drafted turn by turn from what the sweep finds. One window cannot hold both, and the memo degrades whenever the sweep is active. Which context-engineering technique fits, and why don't the others?

- **A.** Tool-output pruning — clear each vibration-log read once its immediate use has passed, which is lossless because the agent can re-call the tool and cheap because no LLM call is needed to do it.
- **B.** Compaction — summarize older history through an LLM call, keeping recent exchanges and key decisions, so both workstreams continue in the same window at reduced fidelity.
- **C.** Delegate the sweep to a **subagent**: it starts with a fresh context window carrying no parent history and no accumulated tool output, and only its final response returns to the parent as a tool result, so the memo's window never holds the sweep's intermediate reads at all.
- **D.** Move the session to a 1M-token model and raise the configured context cap, since the two workstreams only conflict because of how much window is available.

Q16

A satellite-imagery firm holds its entire compliance posture inside its own AWS account, and its security team's stated condition is that no request data may be processed outside that AWS boundary. The procurement lead prefers **Claude Platform on AWS**, because it bills through the existing AWS account and uses Anthropic's own model IDs and lifecycle. What should the engineer point out?

- **A.** Both options place inference inside the customer's AWS account, so against this requirement the choice is cosmetic — the only real difference is the model ID format, first-party on one and `anthropic.`-prefixed on the other.

- **B.** Claude in Amazon Bedrock is the option that runs on Anthropic-operated infrastructure, so the procurement preference happens to be the right one for an in-boundary requirement.
- **C.** Neither option satisfies a requirement stated that strictly; an in-boundary guarantee is only available through Vertex AI's regional endpoints or the legacy Bedrock surface.
- **D.** On Claude Platform on AWS the customer's AWS account supplies billing and access under Anthropic's identity and terms, but **inference is Anthropic-operated, outside the AWS boundary**; Claude in Amazon Bedrock is the option where data stays inside the customer's AWS boundary, serving the Messages API at `/anthropic/v1/messages` with broad feature parity. The requirement selects Bedrock.

Q17

An online notary service runs identity-check reasoning on Haiku 4.5 with extended thinking enabled and archives every session for audit. To hold the archive down, the persistence layer condenses each `thinking` block to a one-paragraph summary — and the session-resume path replays conversation history out of that same archive. Resumed sessions are now rejected by the API. Why?

- **A.** A `thinking` block must be passed back **unchanged** in later turns: its signature verifies that it hasn't been edited, so any modification — including a summary — breaks the signature and the request is rejected. Redacted thinking blocks follow the same rule despite being encrypted and unreadable, so the condensed text belongs in the audit archive while the resume path replays the original blocks verbatim.
- **B.** Thinking content is omitted from responses by default on the newest models, so the persistence layer has been summarizing blocks that were never returned in the first place; requesting summarized display explicitly is what repairs the resume path.
- **C.** Thinking blocks may not be replayed at all — the API rejects any assistant turn on resume that still contains one — so the resume path should strip them entirely rather than summarize them.
- **D.** Extended thinking is incompatible with session resumption; moving to adaptive thinking on Sonnet 5, where depth is tuned by `effort`, removes the replayed block from the problem altogether.

Q18

A ferry operator's fleet-maintenance assistant runs today as one agent. The team proposes restructuring it as a supervisor delegating to specialist subagents for hull, propulsion, and licensing. The maintenance manager's objection is about what the restructure does to the **record** of how a recommendation was reached. Which statement describes the architectural tradeoff correctly?

- **A.** Subagents share the supervisor's context window, so nothing leaves the record; the only thing that changes is which system prompt is active for each subtask.
- **B.** Isolation is the structural point: a specialist's intermediate tool calls and reasoning never enter the supervisor's context, which is what keeps the supervisor's window lean and lets each specialist carry deep, narrow instructions and a tool set scoped down to reduce the blast radius of a mistake. The cost is the manager's exact concern — the supervisor is handed a final summary in place of the intermediate work, and each specialist runs its own window and spends its own tokens.
- **C.** Subagents inherit the parent conversation and its files and then hand back a summary, so the full record still exists on the parent side and the restructure is purely a cost decision.
- **D.** The restructure restores step-level observability through standard tooling, because each specialist's turns are logged as their own run — which is why the supervisor pattern is the recommended structure whenever an audit record matters.

Q19

A brewery group has a four-year-old Bedrock integration that calls `InvokeModel` and `Converse`, with model identifiers written as ARNs carrying a version. A new hire who has read that "Bedrock serves the Messages API at `/anthropic/v1/messages` with full model IDs under an `anthropic.` prefix" concludes the

configuration is broken and the deployment is unpinned. What is the accurate picture?

- **A.** The new hire is right: an ARN is a resource address rather than a version pin, so this deployment has been running unpinned for four years and needs a full model ID with the `anthropic.` prefix.
- **B.** ARNs do pin correctly, but `InvokeModel` and `Converse` are the current Bedrock surface for Claude, and the `/anthropic/v1/messages` path the new hire read about belongs to Claude Platform on AWS instead.
- **C.** The brewery is on the **legacy** Bedrock surface — AWS identity and billing, the `InvokeModel` / `Converse` APIs, ARN-versioned model identifiers — where pinning is done through the ARN's version, and that is a valid place for an existing integration not yet migrated. The `/anthropic/v1/messages` path with `anthropic.`-prefixed model IDs describes Claude in Amazon Bedrock, which is the surface they would migrate to.
- **D.** Both descriptions cover one surface: the ARN form is simply the older way of writing the same `anthropic.`-prefixed identifier, so nothing here is legacy and nothing needs to change.

Q20

A sportswear brand submits a 4,000-case eval re-run through the Message Batches API and gets a `batch_id` back. Twenty minutes later nothing has completed, and an engineer opens an incident on the grounds that the API is degraded. What should the team understand?

- **A.** Batch completion time scales with the longest single request in the submission, so the incident is legitimate whenever one case runs unusually long; splitting the set into smaller submissions returns results sooner.
- **B.** Results are released only once every request in the submission has finished, so one stuck case holds the whole set — the remedy is resubmitting as several smaller batches.
- **C.** Twenty minutes already exceeds what 4,000 concurrent synchronous calls would have taken, which shows the batch was the wrong tool and that the discount doesn't apply to eval workloads.
- **D.** Batch jobs can take **up to 24 hours**, and that latency is exactly what is exchanged for the lower per-token cost — the submission model exists for offline pipelines, eval runs, and bulk jobs where nobody is waiting, so twenty minutes without results is expected rather than a fault. Poll for completion and download the results when done.

Q21

A tax preparer's return-review agent has a `PreToolUse` hook that denies any `write_file` outside `/workspace/output`, plus a least-privilege role denying reads of `/secrets` and `~/.aws`. In a red-team exercise, an instruction hidden in a client-supplied PDF persuades the agent to POST a summary of the client's income to an external address using its HTTP-fetch tool. No file was written, no secret was read, and **no hook fired**. Which control addresses this class of gap?

- **A.** OS-level sandboxing, which isolates the agent at the process level rather than the rule level: filesystem isolation confines it to its working directory regardless of what any individual hook permits, and network isolation restricts outbound connections to a named endpoint set regardless of what the identity role allows. Because the OS enforces it rather than application logic, it holds even when a hook is missing, misconfigured, or bypassed.
- **B.** Widen the `PreToolUse` hook so it inspects the fetch tool's destination as well as `write_file` paths, since what failed here is a missing check rather than a missing layer.
- **C.** Tighten the least-privilege role so the HTTP-fetch tool is unavailable on any turn whose context contains third-party content, since least privilege is the control that holds when every other layer fails.
- **D.** Wrap the client PDF so the model treats its contents as data and add a standing instruction never to act on instructions found inside documents, since the exfiltration started life as an indirect injection.

Q22

A wind-turbine servicer runs its work-order assistant on Google Vertex AI against a pinned full model ID. Their upgrade process is a single calendar reminder, set from the retirement date Anthropic published for that model, on the assumption that the platform mirrors that schedule. What is the risk in that assumption?

- **A.** Retirement dates attach only to aliases; a pinned full model ID remains callable indefinitely, so the reminder is tracking a date that will never apply to this deployment.
- **B.** Partner platform retirement dates **differ** from Anthropic's own schedule, so the model's availability window has to be tracked against Vertex's schedule rather than Anthropic's announcements — the same caution applies on Bedrock — and the eventual move still has to be gated on the eval suite against the pinned baseline score.
- **C.** The risk runs the other way: partner platforms retire a model later than Anthropic does in every case, so the reminder fires early and forces a migration the deployment did not yet need.
- **D.** Vertex names the model in the endpoint URL rather than the request body, so the pin lives in the URL and platform retirement is absorbed by Google's own version aliasing rather than by your configuration.

Q23

A campground chain's booking repo carries a CLAUDE.md rule: never modify anything under `legacy_pricing/` without a migration note. Interactive sessions honor it consistently. When the team delegates a repo-wide audit to the built-in `Explore` subagent and hands the findings to the built-in `Plan` subagent, the resulting plan both proposes edits under `legacy_pricing/` and describes files that were reverted on the current branch as though they were current. What accounts for **both** symptoms at once?

- **A.** CLAUDE.md dilution: the file has grown large enough that the path restriction is a small fraction of what loads, so it carries too little weight to hold.
- **B.** The restriction belongs in a `.claude/rules/` file with a `paths` glob, because a rule written in CLAUDE.md is scoped to the directory that file sits in and never reached the delegated work.
- **C.** The built-in `Explore` and `Plan` subagents skip CLAUDE.md and git status entirely — they are optimized for speed, while `general-purpose` loads both — so a project rule silently not applying to a delegated task and a plan built on stale file state are the same cause.
- **D.** `plan` mode blocks edits until the plan is approved, so the proposed edits were never a real violation, and the stale file descriptions come from compaction dropping the git status captured earlier in the session.

Q24

A film archive's cataloguing assistant must return a fixed condition-report block: four labeled lines in a set order, with specified units. The system prompt describes that layout in careful prose — line order, label wording, units — and the assistant's condition assessments are accurate with every required piece of information present, but on roughly a third of reels it returns **its own** labels and ordering. Two rounds of firmer wording have not moved it. Which of the four prompting techniques is missing?

- **A.** An output constraint, since the prompt never specified the form, the field names, or where the response should stop.
- **B.** A more specific system prompt, since the behavioral contract is too vague to hold across a long cataloguing session.
- **C.** A constraint covering the variant that breaks, since the prompt was validated against a narrow set of reels with no rule for the ones that fail.
- **D.** Few-shot examples: "the task is right but the structure is invented" is the failure signature that points at missing examples, because a description alone cannot pin down an exact structure while an example shows it directly — with XML tags marking where each example starts and ends so Claude doesn't read them as part of the live instruction.

Q25

An animal shelter's adoption-matching service pins `claude-sonnet-5` in its configuration. A reviewer flags the line as an unpinned alias on the grounds that it carries **no date suffix**, and asks for a dated ID instead. What is the correct response?

- **A.** The reviewer is right: every pinned Claude model ID carries a date suffix, so a dateless identifier is by definition the moving-alias form and the service has been exposed to silent version changes.
- **B.** The reviewer is right about the risk but wrong about the remedy — the way to pin on the current generation is to keep the identifier and add the `anthropic.` prefix, which is what resolves it to a fixed snapshot.
- **C.** From the Claude 4.6 generation onward the model ID alone pins to a specific snapshot in a dateless-but-still-fixed format; the aliases are the bare tier names like `sonnet` and `opus`, which resolve to a recommended version that moves over time and can differ by platform. Only earlier models need the ID plus a date suffix.
- **D.** The identifier format doesn't determine pinning at all: pinning is a platform-level setting, so on the first-party API you pin by sending an `anthropic-version` header rather than through the model string.

Q26

A snowplow fleet operator's dispatch assistant is quiet all summer and saturated during a storm. Every API call is wrapped in the team's own loop that retries any non-200 response five times in quick succession, sitting on top of the SDK client's built-in retries. During the first big storm the 429s got steadily worse rather than clearing, and a **400** caused by a malformed tool block took an afternoon to find because it was buried under identical repeats. What is the correct fix?

- **A.** Classify each failure first — 429 and 529 are retrievable, a 400 is terminal and an identical request fails identically — and check what the SDK already retries, because a hand-rolled loop on top of the client's automatic retries multiplies attempts against the same rate limit instead of capping them; honor `retry-after` on the responses that carry it.
- **B.** Keep retrying every status code but replace the tight loop with exponential backoff and jitter, since adding delay is what makes a retry safe regardless of which error came back.
- **C.** Delete the application's loop entirely and raise every failure to the caller, since the SDK's automatic retries already cover the full set of status codes including authentication and bad-request failures.
- **D.** Treat both classes as retrievable, because the documented default when a failure's cause is unclear is to retry rather than risk dropping a dispatch that would have succeeded.

Q27

A cinema chain generates listings blurbs against a fully pinned model ID, and promotes a new pin only after sending it to a traffic slice and comparing the result with the baseline eval score. The new pin has just cleared that comparison, and an engineer proposes deleting the **previous** model ID constant in the same commit to keep the configuration tidy. What does that specifically give up?

- **A.** Nothing — a pinned ID is a fixed snapshot, so behavior cannot shift after promotion and the old constant is dead configuration by definition.
- **B.** Rollback. Retaining the prior pinned version is what makes a regression discovered after promotion a rollback rather than a hotfix, and an eval slice cannot surface every behavior shift the wider traffic will find.
- **C.** Nothing material, because rolling back means switching the model field to the `sonnet` alias, which resolves to a recommended version already known to work in production.
- **D.** The eval baseline, since the baseline score is recorded against the model ID it was measured on and the next promotion has nothing left to compare against.

Q28

A comic-book publisher's editorial tooling runs a continuity pass in which each step consumes the previous one's output: script normalization, then a continuity check against earlier issues, then dialogue trimming, then balloon placement. Hoping for a rough four-fold speedup, the team fanned the pass out across four subagents under a lead. Latency improved slightly, the monthly bill roughly tripled, and reviewers could not tell the output apart from the single-agent version. What is the correct reading?

- **A.** The fan-out is sound but under-parallelized: splitting the pass across more subagents so each owns a smaller slice is what recovers the expected speedup and amortizes the lead's synthesis cost.
- **B.** Context isolation is the problem — each worker started without the parent's history, so the lead had to re-explain the issue's continuity, and passing the parent's full context into every subagent fixes both the cost and the quality result.
- **C.** Orchestrator-workers runs at roughly 15x the token cost of a single-agent interaction because a lead, several workers, and a synthesis pass each carry their own input and output tokens, and that multiplier only buys something when the work genuinely decomposes into independent parts explored in parallel. Tightly-coupled work where each step depends on the last — coding is the standard counterexample — is where subagents mostly wait on each other, so moving the pass back to a single agent with the same context is the fix.
- **D.** The multiplier is fixed overhead of using subagents at all, so the same tripling would have appeared on genuinely parallel work; the lever that brings it down is a lower `effort` setting on the workers rather than a change of pattern.

Q29

A pension administrator is building a long-running agent that reads whole scheme rulebooks alongside decades of member correspondence in a single session, and has selected `claude-fable-5`. A new engineer asks which header enables thinking and how large a history the session can hold before trimming. Which pair of properties describes this tier correctly?

- **A.** A 200K-token context window, with reasoning enabled per request via `thinking.type: "enabled"` and depth set by the model.
- **B.** A 1M-token context window, with adaptive thinking off by default and switched on with `thinking.type: "enabled"` when a task warrants it.
- **C.** A 1M-token context window, with extended thinking always on and its depth tuned through `budget_tokens`.
- **D.** A 1M-token context window, with adaptive thinking **always on** — the model decides how much to think, tuned through `effort` rather than an enable flag or a token budget.

Q30

An escrow agency's document-summarization service is built on the Python SDK, and the engineering lead is setting the scope of its code review. The author argues that because the SDK "owns the HTTP layer," ordinary REST concerns belong to Anthropic rather than to this codebase. Which assessment should the review be run against?

- **A.** The Messages API is a REST API returning JSON, so the service inherits ordinary REST concerns whichever client it uses: idempotency of the surrounding escrow operation, status-code handling along the retrievable/terminal split, and validation of request and response schemas.
- **B.** Only rate-limit handling carries over to the application; idempotency and schema validation are properties of the SDK's request construction and response parsing and are settled there.
- **C.** None of them apply at the application level, because the SDK's typed response objects guarantee the response schema and its retry logic covers every status code the API can return.

- **D.** They apply, but only to services that call the endpoint over raw HTTP; through an SDK a call is a local function invocation whose failures surface as language-level exceptions rather than HTTP statuses.

Q31

A glassworks connects Claude Code to an MCP server named `mes` that exposes a dozen read-only production tools plus `mcp__mes__set_furnace_setpoint`. The committed project settings allow the whole server so shift engineers aren't prompted on every reading they pull, but the setpoint tool must **never** execute without a human confirming it. What configuration achieves that?

- **A.** Drop the server-wide allow and write an individual allow rule for each read-only tool, since a permission rule can only target a whole MCP server or nothing.
- **B.** Keep the server-wide allow and add a deny rule on `mcp__mes__set_furnace_setpoint` — MCP permission rules target one tool on a server at `mcp__server__tool` granularity, and a deny on a single tool overrides an allow on the whole server.
- **C.** Switch the shift engineers' sessions to `dontAsk` mode, which auto-denies state-mutating tools while leaving read-only commands approved.
- **D.** Put the deny in `.claude/settings.local.json`, because a deny only outranks an allow when both sit in the same scope, so the rule has to live above the committed project file to take effect.

Q32

A drone-survey firm wraps the Agent SDK's `query()` in its own harness and currently logs only the last text it sees. Auditors now want, per flight-plan run, the session ID, the token usage and cost, and a recorded reason the run ended. Which description of the message sequence should the logger be written against?

- **A.** A `ResultMessage` arrives first carrying the session ID, followed by `AssistantMessage` objects as the work proceeds; hitting `max_turns` raises an exception mid-loop, so the reason for stopping has to be caught rather than logged.
- **B.** A `SystemMessage` opens the run and then one `ResultMessage` is emitted per turn, with tool results delivered as further `SystemMessage` events.
- **C.** The session opens with a `SystemMessage` (`subtype: "init"`) carrying session metadata, then cycles `AssistantMessage` (text, tool-call requests, or both) and `UserMessage` tool results until Claude replies with no tool calls, closing with a final `AssistantMessage` and then a `ResultMessage` carrying final text, token usage, cost, and session ID — whose `subtype` names why it ended (`success`, `error_max_turns`, `error_max_budget_usd`).
- **D.** Only `AssistantMessage` objects are yielded during the run; session ID, cost, and stop reason are read back off the `CLaudeAgentOptions` object once the iterator is exhausted.

Q33

An elevator maintenance company triages faults through a five-step prompt chain: parse the report, look up the unit, classify the fault, pick a parts kit, draft the dispatch note. Reports now arrive as free-form technician voice notes with wildly varying telemetry exports attached, and the chain breaks whenever an input doesn't fit the predetermined path — the team has added eleven branches this quarter. They propose rebuilding it as an agent with the same tools. How should a reviewer classify and judge that proposal?

- **A.** As small-scale refactoring: what actually changes is the wording of the step prompts and the tool descriptions, the same class of change as splitting an over-broad tool into two narrower ones.
- **B.** Not a refactor at all — the observable behavior changes, so it is a new feature, and the existing eval suite and its baseline score no longer apply to it.

- **C.** As large-scale refactoring pointed the wrong way: a workflow that breaks on unexpected input needs its branch coverage completed, because an agent surrenders the step-level guardrails that a safety-critical dispatch decision depends on.
- **D.** As large-scale refactoring, and correctly aimed: migrating a workflow architecture to an agent architecture as requirements shift from predictable to open-ended is the named large-scale case, and a workflow that breaks the moment input deviates from the predetermined path is the documented failure mode of a workflow where an agent is needed — the accepted tradeoff being non-determinism bounded by the registered toolset and transcript-level rather than step-level observability.

Q34

A sheet-music publisher's assistant serves two very different request types against one top-tier model: high-volume catalogue metadata lookups, where quality has never been in question, and a much smaller stream of arrangement-difficulty analyses that genuinely need the capability. Spend is the complaint. Which change matches the guidance, and what is the stated condition on it?

- **A.** Add a cheap classification call that reads a request-level signal — task type, input length, or a difficulty label — and send the bulk of traffic to a default tier while routing only the hard slice to a larger model; the classification call only pays for itself because this traffic mix is genuinely heterogeneous, and on uniform traffic you skip the router and pin one model.
- **B.** Keep the single top-tier model and enable prompt caching on the shared system prompt and tool definitions, since caching is the cost lever for mixed traffic while routing is a latency technique.
- **C.** Send everything to the cheapest tier first and escalate to the larger model whenever the cheap answer fails validation, which removes the need for a classifier entirely.
- **D.** Split the assistant into two applications, each with a hardcoded model, and let the calling client choose — a classifier adds a model call to every request and so can never pay for itself.

Q35

An olive-oil bottler's scheduling agent calls `schedule_bottling_run(line_id, litres, blend_code)` inside a tool-use loop. Occasionally Claude passes `litres` as a string or omits `blend_code`, and the function either throws mid-loop or books the wrong blend. The system prompt already instructs it to supply all three fields exactly as specified. Which mechanism moves that from a request to a **guarantee**?

- **A.** JSON outputs via `output_config.format` with a `json_schema` naming the three fields, which constrains the tokens the model is allowed to emit.
- **B.** `strict: true` on the tool definition — strict tool use applies constrained decoding to the arguments Claude passes to your tools, validated against the tool's input schema before your code runs, which is the mechanism for an agentic loop where a malformed argument crashes the function or triggers the wrong action.
- **C.** Marking all three fields `required` in `input_schema`, so the API rejects any call that arrives missing one of them before the function is reached.
- **D.** A `PreToolUse` hook that inspects the arguments and exits with code 2 to block the call, writing the reason to stderr so the agent can correct itself.

Q36

A laundry franchise's `create_pickup` tool is consumed by three internal applications and re-exposed through an MCP server the franchisees' portal connects to. In one pull request an engineer renames `bag_count` to `item_count` and marks a previously optional `route_code` as required, describing it in review as "just the JSON blob we hand the model." Which framing should the reviewer apply?

- **A.** Schemas are model-facing configuration, so the only question the review has to settle is whether Claude still selects the tool reliably; caller compatibility is the API gateway's concern.

- **B.** The change is safe as long as the eval suite still passes, since the eval exercises the tool end to end and would fail on any break a caller could see.
- **C.** Tool input/output schemas are part of the application's **API contract** and a schema change gets the same discipline as a public API's breaking-change policy, because it can silently break every caller relying on the old shape and no compiler will catch it — which is why code review for a Claude application explicitly covers prompt and tool-schema diffs, not just surrounding logic.
- **D.** The rename is a breaking change but the new `required` field is not, since marking a field required governs whether Claude fills it in and has no effect on what existing callers send.

Q37

A marina's berth-management agent runs under a least-privilege role: write access to `/workspace/output` only, read access to `/workspace/input`, explicit denies on `/etc` and `/secrets`, and a `PreToolUse` hook that blocks writes outside the output path. The role itself is declared in a YAML file that lives in the repository the agent is allowed to edit as part of its normal work. A security reviewer flags this as the gap that undoes the rest. Why?

- **A.** The role is written as allow-plus-deny rather than deny-by-default, so any path neither allowed nor denied is implicitly writable and the YAML file falls into that space.
- **B.** Hooks execute in the application process rather than inside the model's context, so the agent can reason about the rules and route around them; the fix is to stop restating the hook's rules in the prompt.
- **C.** The uncovered surface is network egress — a hook checking writes says nothing about outbound connections — and OS-level network isolation is what closes it.
- **D.** Anything that can modify the agent's own auth or role configuration can effectively act with that identity, so the role file is itself a privileged target: editing the role is a privileged action and belongs behind the same protection as the secrets, rather than sitting inside the agent's own writable scope.

Q38

A prosthetics manufacturer prototyped a socket-fitting agent on the Agent SDK and now wants to move it to Managed Agents: runs take tens of minutes, and nobody wants to build the sandbox or the execution layer. Sessions carry patient scan data and clinical notes, and the hospital contract requires a HIPAA Business Associate Agreement and Zero Data Retention. Operationally the fit is exact. What is the correct call?

- **A.** Managed Agents is ruled out regardless of operational fit — its sessions are stateful and stored server-side, and it is not currently eligible for Zero Data Retention or a HIPAA BAA — so the workload stays on the Agent SDK or a raw Messages API loop running on a BAA-covered configuration.
- **B.** Adopt it, sending the `managed-agents-2026-04-01` beta header with a key issued under the organization's existing ZDR agreement, which extends that agreement's coverage to the managed sessions.
- **C.** Adopt it, because ZDR and BAA coverage attach to the model rather than to the surface, so pinning a BAA-eligible model keeps the configuration compliant wherever the loop runs.
- **D.** Adopt it, and treat the migration as a lift-and-shift: the prototype's Agent SDK configuration exports directly into a managed agent definition, so nothing about the compliance posture changes.

Q39

A quarry's engineering team runs one agent for two jobs. The first lists which survey files changed since the last shift. The second reconciles bench geometry, vibration limits, and haul-road access across a multi-step blast-sequence analysis. Both currently run at `effort: medium`. Which assignment follows the documented effort levels?

- **A.** Both at `high`, since `high` is the documented level for complex agentic tasks and `xhigh` and `max` exist only inside Claude Code rather than at the API level.

- **B.** `low` for the file listing and `xhigh` or `max` for the blast-sequence analysis — `xhigh` is extended reasoning depth for complex coding and agentic tasks, `max` is maximum depth for multi-step problems needing deep analysis, and reasoning depth is wasted on lookups and listing.
- **C.** `max` for both, because one agent should run a single effort level and additional reasoning depth never costs correctness.
- **D.** `medium` for both, raising `budget_tokens` on the analysis call, which is the current control for how deeply the model reasons.

Q40

A commercial apiary's platform team locks tool permissions for its hive-telemetry repository through enterprise managed settings. A developer adds an allow rule in `.claude/settings.local.json` for a tool the committed project settings never mention, and separately assumes their `~/ .claude/settings.json` is ignored outright because a project settings file exists. Which description of the scopes is correct?

- **A.** Precedence runs user > project > local > CLI > managed, so the developer's personal file wins and managed settings act as the fallback when nothing else defines a key.
- **B.** Precedence runs managed > CLI > local > project > user and every key follows it uniformly: the highest-precedence scope that defines a key replaces the lower scopes' value outright.
- **C.** Precedence runs managed > CLI > local > project > user, but **permissions are the exception** — they accumulate across scopes rather than overriding, so the local allow rule adds to the project's rules and the user-scope rules still apply; a managed deny stays the most durable control, since no user or project file can remove it and a deny always beats an allow.
- **D.** Precedence runs managed > CLI > local > project > user for ordinary settings and permissions accumulate, but managed scope only supplies defaults, so a project-scope allow can widen a managed deny for that one repository.

Q41

A dog-grooming chain's booking service calls the Messages API directly and wants to reach its grooming-inventory MCP server, which today runs as a local stdio process next to each groomer's Claude Code install. The server exposes about forty tools; the booking service needs four of them, and the team wants the remaining schemas to stay off requests that don't use them. What does the API MCP connector actually support here?

- **A.** Register the stdio server in `mcp_toolset` and set `defer_loading: true` in `default_config`; transport is a client-side detail the connector abstracts away.
- **B.** Register the server and switch the unused tools off with `enabled: false`; `defer_loading` is not a connector setting but a Claude Code permission behavior.
- **C.** Register the server via `mcp_toolset` behind the `mcp-client-2025-11-20` beta header, but accept that `default_config` applies to every tool on the server uniformly, since per-tool overrides are not part of the connector.
- **D.** The `mcp_toolset` object does exactly this — a `default_config` applied to every tool with per-tool overrides in `configs` keyed by tool name, `defer_loading` to delay a definition until the model needs it and `enabled` to expose only a subset, behind the `mcp-client-2025-11-20` beta header — but the connector supports **remote HTTP servers only**, so the stdio server has to be moved to HTTP transport or stay behind Claude Desktop or Claude Code as its client.

Q42

A mattress retailer's engineering lead wants Claude Code to trace how a stacked-discount rule flows through the pricing service and come back with a written description of the edits it would make, with a hard guarantee that nothing is written or executed until a human has read that description. Which permission

mode is built for exactly this?

- **A.** `plan` — it holds the session in the explore phase, auto-approving reads so Claude can research and propose while blocking every edit and command until the plan is approved.
- **B.** `default` — it auto-approves reads and gates edits and shell commands, which makes it the mode designed to hold a session at the planning stage.
- **C.** `acceptEdits` with deny rules on the pricing module, so routine edits elsewhere proceed while the sensitive path stays gated behind a prompt.
- **D.** `dontAsk` with the read tools on the allow list, since everything off the list is auto-denied and no edit can slip through on a mis-click.

Q43

A funeral home group's Agent SDK service dispatches subagents to pull records from several registries in parallel. Two requirements land in the same sprint: the compliance file needs the **full** transcript of every session retained before the window is condensed, and operations wants a per-run count of which subagent dispatches started and which finished. Which hooks cover these, and what do they cost in context?

- **A.** `PreToolUse` to capture the transcript and `PostToolUse` to detect completions, since a subagent dispatch is a tool call like any other; both add their output to the model's context.
- **B.** `PreCompact` fires before context compaction and is where the full transcript gets archived before it is summarized away, while `SubagentStart` and `SubagentStop` fire as a subagent spawns and completes and are the documented way to track and aggregate parallel results — and because hooks run in the application process rather than the model's context window, neither consumes tokens.
- **C.** `Stop` for the transcript, because compaction happens at the end of a turn, and `SubagentStop` alone for the counts, because no corresponding start event is emitted.
- **D.** `SessionEnd` for the transcript and `Notification` for subagent lifecycle events, with the transcript cost billed as input tokens on the next request.

Q44

A hydroelectric plant's operations analysts have spent months refining a set of standing instructions inside a claude.ai Project that drafts outage reports in the regulator's preferred structure. The engineering team now wants that same behavior in a service that calls the API, and the compliance officer wants the wording itself reviewable and version-controlled. What is the accurate picture?

- **A.** Export the Project's custom instructions to a `CLAUDE.md` and the API service inherits them, since `CLAUDE.md` is the file-based form of the same persistence mechanism.
- **B.** Persistent instructions are organization-scoped once set, so the API service picks up the Project instructions automatically and only the wording itself still needs versioning.
- **C.** In Claude Desktop and claude.ai, persistent instructions come from custom instructions and Projects and are user- or workspace-scoped rather than file-based, so there is no artifact to review; on the API the `system` parameter is fully under the application's control with no implicit persistence, so the wording has to be re-sent on every call and tracked as production configuration — version control, changelog, rollback — like any other prompt.
- **D.** The Project already is the versioned artifact, because claude.ai keeps a revision history per Project that an API request can reference by ID instead of re-sending the text.

Q45

A fish hatchery's water-quality agent was capped at 40K tokens as a deliberate cost control, far under the model's own ceiling. Against development fixtures, tool results averaged around 800 tokens and a full 20-turn session used roughly 18K. In production the agent starts selecting obviously wrong tools and returning

incomplete analyses at **the same point in every run**, roughly eight turns in, and two engineers are drafting tighter tool descriptions. What is the actual diagnosis?

- **A.** The tool descriptions genuinely overlap and the failure surfaces late only because later turns carry more ambiguous requests; adding an exclusion sentence naming when *not* to call each tool is the documented fix.
- **B.** The cap is causing generation to hit the ceiling mid-response, so runs return partial output with stop reason `model_context_window_exceeded`; raising the cap and handling that stop reason resolves it.
- **C.** Failed sensor queries are returning empty results without `is_error: true`, so the agent treats missing data as valid and reasons on top of it, which reads as erratic tool choice.
- **D.** Accumulated, never-pruned tool outputs have crowded out the system prompt and early instructions that tell the agent which tool to use next — the symptom of context overflow is frequently misread as a tool-selection or schema problem, and degradation after a consistent number of turns is the tell. The cap was validated against dev fixtures rather than the largest realistic production input, so the fix is to measure a real tool result against the budget and prune or compact proactively.

Q46

A toll-road operator building an incident-review assistant under a state transportation contract is told the workload requires FedRAMP High. Procurement has already bought Claude Enterprise through the AWS Marketplace on the assumption that "it's AWS, so it's in scope," and the team's staging environment points at the commercial endpoint while production would point at the authorized one. What should the engineer tell the review?

- **A.** Claude Enterprise on the AWS Marketplace is explicitly **not** FedRAMP authorized; the authorized routes are Claude for Government (FedRAMP High via Palantir Federal Cloud), Claude via Amazon Bedrock GovCloud (FedRAMP High, DoD IL4/5), and Claude via Vertex AI Assured Workloads — and the staging path counts, because any code path hitting an endpoint outside the authorized environment at the required impact level is ruled out, not just the production one.
- **B.** Marketplace procurement is a billing arrangement, so authorization follows the underlying service — any Bedrock access pattern therefore satisfies FedRAMP High, and only the staging endpoint needs changing.
- **C.** The Marketplace purchase is acceptable for production, but staging has to move to GovCloud because dev and test traffic carries the same incident data as production.
- **D.** Only the direct Anthropic API is authorized at FedRAMP High; the cloud-mediated routes address data residency requirements rather than federal authorization, so the Marketplace purchase and the staging path are both beside the point.

Q47

A locksmith network is choosing between two deployment platforms for its dispatch assistant. An engineer benchmarks both from their own laptop with a short test prompt, reports a difference of a few milliseconds, and calls latency a tie. The customer's call centre and all its data sit in another country. What is the correction?

- **A.** Latency is not a meaningful axis for comparing platforms, since per-token rates are broadly aligned across them; decide on compliance and cost and treat latency as settled.
- **B.** Latency has to be measured from the **customer's actual region against their actual payload** — a measurement from your own laptop hides the round-trip penalty that only appears once the workload runs where the customer really is, and a short test prompt doesn't represent a real dispatch request.
- **C.** Re-run the same laptop benchmark with streaming enabled, since streaming determines what latency the operator perceives and removes the dependence on where the measurement is taken.

- **D.** Measure from the customer's region but keep the short prompt, since payload size drives cost rather than latency and holding it constant is what keeps the two platforms comparable.

Q48

A ski-lift manufacturer's inspection assistant sends a long, stable system prompt plus a large tool schema ahead of a conversation that grows across a whole inspection. One engineer wants caching that keeps working as the conversation lengthens without anyone hand-placing markers; another wants a breakpoint immediately after the tool definitions. Which description of the two mechanisms is correct?

- **A.** Automatic caching marks every cacheable block, which is why it is capped at four blocks per request; explicit breakpoints exist to lift that cap for larger prefixes.
- **B.** Automatic caching is the only mechanism that survives a growing conversation, because an explicit breakpoint is invalidated as soon as a new message is appended after it.
- **C.** Automatic caching is a single top-level `cache_control` field: the system applies the breakpoint to the last cacheable block and slides it forward as the conversation grows. Explicit breakpoints put `cache_control` on individual content blocks — up to four per request, subject to a 20-block lookback and a per-block minimum of roughly 1,024 tokens — and since requests process in the order tools → system prompt → messages, a breakpoint after the tool definitions caches them while leaving the messages dynamic.
- **D.** They are the same field with different lifetimes: automatic caching implies the 5-minute TTL, and setting explicit breakpoints is what makes `ttl: "1h"` available.

Q49

A stage-lighting rental company answers customer questions over a library of rig manuals and hire agreements that is revised daily. One proposal pre-builds an embedding index over chunked documents; the other gives the agent MCP tools and lets it search and fetch at the moment of need. Which statement characterizes the difference correctly?

- **A.** Both approaches find a relevant slice and generate from it — the difference is **when** the relevant-slice decision happens: classical RAG builds an embedding index over chunked material ahead of time and matches the query's embedding at request time, while agentic search skips the index and has the model search and fetch live, which is what MCP tool search and Claude.ai Projects' document surfacing both do. Either way request cost stays flat as the library grows, and poorly named or badly organized material degrades both.
- **B.** Agentic search puts the whole corpus into context each turn, so it scales on quality but not on cost; classical RAG is the only one of the two whose request cost stays flat as the library grows.
- **C.** Classical RAG grades relevance with a model call per chunk at request time, which makes agentic search the cheaper option — the tradeoff being that agentic search cannot see documents added after the session started.
- **D.** The two are not comparable: RAG is a retrieval technique, while agentic search is a permission setting that governs which documents a subagent is allowed to read.

Q50

A seed distributor runs its entire platform on AWS and wants an open-source, model-driven agent framework with deep Bedrock integration for a new inventory-forecasting agent. Which framework fits, and how should the team frame it against the Claude Agent SDK?

- **A.** LangGraph — graph-based orchestration, and the AWS-aligned choice because each node in the graph maps to a single Bedrock invocation.
- **B.** PydanticAI — Python-first type safety, and the natural fit in an AWS stack because Bedrock's request shape requires schema validation on every call.

- **C.** The Claude Agent SDK, because cross-provider frameworks cannot run their loops against Bedrock-hosted models at all.
- **D.** Strands Agents — AWS's open-source, model-driven framework, deeply integrated with Amazon Bedrock and the natural choice inside an AWS-centric stack. The framing: the Claude Agent SDK is a provider-native primitive with the loop, tool execution, MCP integration, and prompt caching built in and optimized for Claude, while Strands, LangGraph, and PydanticAI are independent cross-provider frameworks you reach for when you need a pattern the native SDK doesn't provide — not because the SDK is weaker at the core agent loop.

Section C — Answer Key and Explanations

Q1 — Answer: A

- **Why A is correct:** A solution architecture maps functional requirements *onto* infrastructure choices — "summarize 10,000 documents overnight, cost-sensitive" is what forces the Batch API; "respond to a live chat user" is what forces streaming. The requirement is the input to that mapping, so pinning the behaviors first is what makes the infrastructure choice defensible. Getting the mapping backwards — picking the infrastructure pattern before the actual requirement is nailed down — is named as the root cause behind most of the "why is this slow/expensive" production complaints these systems generate.
- **Why not B:** Infrastructure requirements are mostly *not* stated in the business problem; they are derived by asking what it implies, which can't happen before the functional behaviors exist.
- **Why not C:** Functional requirements are captured in the requirements phase and drive the platform choice at design; they are not derived from a platform already selected.
- **Why not D:** Model tier is chosen empirically against an eval, but it doesn't determine the deployment pattern, and the requirement still has to exist before either decision.
- **Difficulty:** Medium
- **Domain:** Applications and Integration
- **Tag:** `apps.requirements/functional-to-infrastructure`
- **Revise:** `2_applications_and_integration.md` → Understanding Requirements

Q2 — Answer: B

- **Why B is correct:** Parallelization comes in two named forms. *Sectioning* runs independent subtasks simultaneously and buys speed; *voting* runs multiple attempts at the same task and buys confidence through diverse perspectives. The stem describes one judgment per manifest, several independent reads of the same input, and a confidence signal as the goal — that is voting by definition, not a decomposition problem.
- **Why not A:** Sectioning requires independent subtasks; the team explicitly wants repeated reads of the whole manifest, not separate parts of it.
- **Why not C:** Evaluator-optimizer is a generate-then-critique refinement loop, which improves one answer iteratively rather than aggregating several independent attempts.
- **Why not D:** Orchestrator-workers exists for tasks whose subtask structure can't be predicted in advance; here the task is fixed and known before the run starts.
- **Difficulty:** Medium
- **Domain:** Agents and Workflows
- **Tag:** `agents.patterns/parallelization-voting-vs-sectioning`
- **Revise:** `1_agents_and_workflows.md` → The five named workflow patterns

Q3 — Answer: C

- **Why C is correct:** Claude reads tokens, not characters, and the characters-per-token ratio depends on the model's tokenizer, which changes across model generations. Fable 5 uses the tokenizer introduced with Opus 4.7, which produces roughly 30% more tokens than pre-4.7 models for the same text — so a constant calibrated on the older generation now underestimates every log. The rule is never to hardcode a chars-per-token constant; confirm current tokenizer behavior at build time instead.
- **Why not A:** Fable 5 carries a 1M-token context window, so a shrinking window is not what changed; and raising the allowance would leave the same broken estimate in place.

- **Why not B:** Tool definitions and tool results have always drawn on the same shared context budget as the prompt and the response — that is not a change introduced by the migration.
- **Why not D:** Tokenization is a model property, not a fixed API constant, which is exactly why the ratio moved when the model did.
- **Difficulty:** Medium
- **Domain:** Model Selection and Optimization
- **Tag:** `mso.llm-fundamentals/tokenizer-across-generations`
- **Revise:** `4_model_selection_prompting_context.md` → LLM Fundamentals

Q4 — Answer: D

- **Why D is correct:** Eval suites are built as part of the **build** phase, not as an afterthought. That placement is load-bearing twice over: the same suite is the artifact that gates the deploy phase (a new version has to clear it against its pinned baseline score before promotion), and it's what makes iterate → operate tractable once the system is live, because a model or prompt change gets validated against a fixed set of graded cases rather than re-spot-checked by hand.
- **Why not A:** The test phase is where the suite is *run* alongside unit, integration, and end-to-end checks; scheduling its construction there is what leaves the build phase shipping code nothing has graded.
- **Why not B:** Production findings feed back through the iterate phase and add cases, but waiting for them means the deploy gate has no baseline to compare against on day one.
- **Why not C:** The requirements record and the eval suite are different artifacts — specific functional requirements *become* lines in an eval, which is not the same as the eval replacing the record.
- **Difficulty:** Easy
- **Domain:** Applications and Integration
- **Tag:** `apps.lifecycle/eval-suite-in-build-phase`
- **Revise:** `2_applications_and_integration.md` → Systems Life Cycle

Q5 — Answer: A

- **Why A is correct:** Context engineering is the superset of prompt engineering: prompt engineering is about *what you say*, while context engineering is about curating the full set of tokens available at each inference call — prompt, conversation history, tool definitions, tool outputs, and injected documents — because context is a finite, shared resource. The agency's untracked timetable extracts, tool definitions, and thirty turns of history are exactly the material a tuned system prompt says nothing about.
- **Why not B:** Better wording and better examples are prompt engineering; renaming it doesn't add the curation of history, tool output, and injected documents.
- **Why not C:** The relationship runs the other way — context engineering contains prompt engineering, and it covers far more than the system prompt.
- **Why not D:** Automatic compaction is one technique inside context engineering, not the owner of it; the application decides what enters context in the first place.
- **Difficulty:** Easy
- **Domain:** Prompt and Context Engineering
- **Tag:** `prompt.context/superset-of-prompt-engineering`
- **Revise:** `4_model_selection_prompting_context.md` → Context Engineering

Q6 — Answer: B

- **Why B is correct:** The API enforces a structural pairing: a `tool_use` block must be answered by a `tool_result` in the immediately following user turn, matched by that same `tool_use_id`. When a single turn issues multiple parallel calls, the matching `tool_result` blocks return together — that ID match is

how Claude connects each result to its call. Missing results, non-matching IDs, and out-of-order turns all fail request validation before generation starts, so the application's own code has to produce the sequence correctly on every request.

- **Why not A:** Current models deliberately issue multiple independent `tool_use` blocks in one turn; parallel calls are the default behavior, not a validation error.
- **Why not C:** Preserving the `text` block alongside `tool_use` is a real rule, but dropping it corrupts context for later turns rather than failing validation — the failure here is the deferred `tool_result`.
- **Why not D:** This is validated before generation, so there is no prompt-level recovery; a system-prompt sentence cannot make a structurally invalid request valid.
- **Difficulty:** Medium
- **Domain:** Applications and Integration
- **Tag:** `apps.api-mechanics/tool-result-pairing`
- **Revise:** `2_applications_and_integration.md` → Claude API Mechanics

Q7 — Answer: C

- **Why C is correct:** In an `input_schema`, a field should be marked `required` only when the call is meaningless without it. Marking everything required removes Claude's option to omit a value it doesn't have, which forces it to fabricate one — exactly the invented `dealer_id` and `installation_date` in the logs. Fields that have sensible defaults, or where absence itself carries meaning, are left optional, and that alone removes the pressure to guess.
- **Why not A:** Strict tool use constrains arguments to be *schema-valid*; an invented dealer ID is schema-valid, so the constraint passes it straight through.
- **Why not B:** The description drives tool *selection*; the requirement flag is what forces a value to be supplied, and a description sentence can't override it.
- **Why not D:** Splitting one coherent capability into two overlapping tools adds a selection problem on top of the schema problem, when relaxing two `required` flags fixes it outright.
- **Difficulty:** Medium
- **Domain:** Tools and MCP
- **Tag:** `tools.implementation/required-fields-and-fabrication`
- **Revise:** `3_claude_code_tools_mcp.md` → Schema anatomy: what Claude actually reads to select a tool

Q8 — Answer: D

- **Why D is correct:** The model doesn't pick a fixed next token — at each step it samples from a probability distribution, and `temperature` only reshapes that distribution. Lower temperature concentrates probability on the most likely tokens, so `temperature: 0` makes output *more repeatable*, but it does not guarantee identical output across calls. That is precisely why exact-text assertions are flaky: the same correct answer can be worded many ways. Assert on the property that must hold, and reach for a model-graded eval when meaning rather than structure is what needs judging.
- **Why not A:** Streaming changes how a response is delivered and reassembled, not what content is generated.
- **Why not B:** Pinning protects you from a behavior shift on a version bump; it does not make sampling deterministic within one snapshot.
- **Why not C:** The 400 on non-default sampling parameters applies to Fable 5, Opus 5, and Sonnet 5 — not Haiku 4.5 — and a 400 is a loud failure, not a silently ignored parameter.
- **Difficulty:** Easy
- **Domain:** Model Selection and Optimization
- **Tag:** `mso.llm-fundamentals/temperature-zero-not-deterministic`

- **Revise:** `4_model_selection_prompting_context.md` → LLM Fundamentals

Q9 — Answer: A

- **Why A is correct:** The `url` source type keeps the image payload out of the request, and in exchange you take on the dependency that the URL is stable, public, and reachable at the moment Claude fetches it. A 60-second pre-signed link is none of those by the time a queued request goes out, which is why the failures cluster on delayed slips — the guidance says to skip `url` for anything behind auth or on short-lived signed URLs. A `base64` block carries the bytes inline for a one-off, and the Files API is the right answer once the same asset is referenced repeatedly.
- **Why not B:** The fetch happens when Claude reads the image, and a longer client timeout does nothing about a link that has already expired.
- **Why not C:** Re-sending on every turn is the `base64` property; the expiry problem here bites on the very first fetch.
- **Why not D:** It's the **Files API** that is beta and unavailable on Bedrock and Vertex — this reverses which source type carries the platform limitation.
- **Difficulty:** Medium
- **Domain:** Applications and Integration
- **Tag:** `apps.api-mechanics/image-url-source-tradeoffs`
- **Revise:** `2_applications_and_integration.md` → Vision

Q10 — Answer: B

- **Why B is correct:** The two attacks aim at different targets: a jailbreak goes after the model's own safety constraints, while prompt injection hijacks *your application's* instructions — typically through content the agent retrieves, like a landowner PDF. The defense shape is the same for both and has two sides: validate and constrain what reaches the model, and limit what the model is allowed to *do* once steered. Defending only the input side leaves an agent that can file permits free to act on whatever it was persuaded to believe.
- **Why not A:** Collapsing them into one attack is exactly the merge the reviewer objects to, and one input-validation layer covers neither target completely.
- **Why not C:** The split is by target, not delivery route, and injected instructions most often arrive through retrieved content rather than the user's own prompt.
- **Why not D:** Anthropic trains the model and runs classifiers, but is explicit that no agent reading untrusted content is fully immune — the application still owns the action-side defense for both threats.
- **Difficulty:** Easy
- **Domain:** Security and Safety
- **Tag:** `sec.appsec/jailbreak-vs-injection`
- **Revise:** `5_eval_debugging_security.md` → The mechanism behind prompt injection

Q11 — Answer: C

- **Why C is correct:** In a streaming call a `tool_use` block does not arrive complete: its input JSON accumulates across multiple `content_block_delta` events, so the block is only safe to act on once the stream has closed and the full `input_json` has been reassembled. A stream that breaks mid-response is classified as a transient failure, and the prescribed handling is to retry the whole request — never to pass a partially accumulated block downstream as if it were complete, because that produces malformed tool inputs and, here, a real tanker sent to a fabricated route.
- **Why not A:** Resuming a broken stream from an event index isn't the described handling; the retrievable classification applies to the request as a whole.

- **Why not B:** `is_error: true` is for reporting a genuine tool execution failure back to Claude, not for laundering an incomplete tool input into a dispatched call.
- **Why not D:** "Default to terminal when unsure" applies when it's unclear whether a retry could work; a mid-response stream break is explicitly the transient case.
- **Difficulty:** Medium
- **Domain:** Applications and Integration
- **Tag:** `apps.api-mechanics/streamed-tool-use-partial-block`
- **Revise:** `2_applications_and_integration.md` → Streaming with tool use — accumulate before you act

Q12 — Answer: D

- **Why D is correct:** Orchestrator-workers is the pattern for complex, variable problems where the subtask structure depends on the specific input: a central LLM breaks the task into subtasks it *can't* predict in advance and delegates each to a worker. That is the agency's shape exactly — the decomposition is dynamic (agent-like) while each translate-then-review piece is scoped and bounded (workflow-like), which is why the pattern is described as the bridge between a pure workflow and a pure agent.
- **Why not A:** Prompt chaining requires the subtasks to be fixed and known up front, which is precisely what inspecting the bundle is needed to discover.
- **Why not B:** A single broad agent gives up the bounded, scoped worker task that each identified piece already is, trading predictability away where it was available for free.
- **Why not C:** Routing picks a handler from a fixed set of categories; it doesn't decompose one job into a variable number of pieces.
- **Difficulty:** Medium
- **Domain:** Agents and Workflows
- **Tag:** `agents.patterns/orchestrator-workers-dynamic-decomposition`
- **Revise:** `1_agents_and_workflows.md` → The five named workflow patterns

Q13 — Answer: A

- **Why A is correct:** The Python SDK exposes `AsyncAnthropic` for non-blocking `async / await` calls, but the TypeScript SDK's standard client is already Promise-based — there is no separate async client class to find, you just `await` the calls. The property worth keeping straight is what async actually buys: concurrency, so your application can do other work while a request is in flight. The request still returns in real time, so neither language's client lowers per-request latency or cost by being async.
- **Why not B:** The TypeScript client is Promise-based out of the box; concurrency needs neither worker threads nor a change of submission model.
- **Why not C:** Streaming is about receiving a response in pieces as it is generated; it is a separate axis from whether the client call blocks.
- **Why not D:** Async never lowered per-call latency in the Python version either, so nothing about the port is slower for lacking the class.
- **Difficulty:** Easy
- **Domain:** Model Selection and Optimization
- **Tag:** `mso.technical/typescript-sdk-no-async-client`
- **Revise:** `4_model_selection_prompting_context.md` → Technical Fundamentals

Q14 — Answer: B

- **Why B is correct:** The tradeoff is decided by whether anyone is waiting. Realtime — synchronous or streaming — is for when a user or a downstream process needs the result now; the Message Batches API is for bulk, offline, latency-tolerant work, where jobs can take up to 24 hours in exchange for a lower per-

token cost. A customer watching a spinner on a rider assignment is the definition of the realtime case, so the discount is not available to this workload at any volume.

- **Why not A:** `custom_id` solves result-to-input matching because batch results return in arbitrary order; it does nothing about turnaround time.
- **Why not C:** 100,000 requests (or 256MB) is the per-submission *maximum*, not a threshold you have to exceed to get the lower rate.
- **Why not D:** Polling is a real part of the batch flow, but the disqualifying problem is the up-to-24-hour turnaround, not the cost of the poll requests.
- **Difficulty:** Easy
- **Domain:** Applications and Integration
- **Tag:** `apps.design/realtime-vs-batch`
- **Revise:** `2_applications_and_integration.md` → Realtime vs. batch — the core tradeoff

Q15 — Answer: C

- **Why C is correct:** Context isolation via subagents is the cleanest way to keep the main agent's context lean while still doing deep, exploratory sub-work: each subagent starts with a fresh window — no parent history, no accumulated tool outputs — and only its *final* response comes back to the parent as a tool result. Pruning and compaction both keep the two workstreams inside the same window, which is the constraint the stem states cannot be met; isolation is the only one of the three that removes the sweep's intermediate material from the memo's window entirely.
- **Why not A:** Pruning clears re-fetchable tool output *once its immediate use has passed*, and the stem says the sweep needs many reads held side by side, so there is nothing to clear while it runs.
- **Why not B:** Compaction is for bloat that can't be cheaply re-fetched — dialogue and reasoning — so here it pays an LLM call to summarize the wrong material and still leaves one window carrying both workstreams.
- **Why not D:** A bigger ceiling delays the collision instead of resolving it; the guidance is to prune, compact, or isolate proactively rather than to buy headroom for unbounded accumulation.
- **Difficulty:** Hard
- **Domain:** Prompt and Context Engineering
- **Tag:** `prompt.context/subagent-isolation-technique`
- **Revise:** `4_model_selection_prompting_context.md` → Context Engineering

Q16 — Answer: D

- **Why D is correct:** The two AWS options differ on exactly the dimension the security team named. Claude Platform on AWS delivers Anthropic's identity and terms *via* the customer's AWS account — convenient for billing and first-party feature parity — but inference is Anthropic-operated and sits **outside** the AWS boundary. Claude in Amazon Bedrock is the option where data stays inside the customer's AWS boundary, serving the Messages API at `/anthropic/v1/messages` with broad feature parity. Compliance is determined by the platform rather than by your code, and for a regulated customer it is pass/fail, not a tradeoff — so the stated requirement decides this.
- **Why not A:** Only one of the two runs inference inside the AWS boundary; the ID format is a downstream consequence, not the substance of the difference.
- **Why not B:** This inverts the two platforms — it is Claude Platform on AWS whose inference is Anthropic-operated.
- **Why not C:** Bedrock satisfies it directly, and the legacy Bedrock surface is a versioning/API distinction rather than a different data boundary.
- **Difficulty:** Hard
- **Domain:** Applications and Integration

- **Tag:** `apps.config/claude-platform-on-aws-vs-bedrock`
- **Revise:** `2_applications_and_integration.md` → Deployment platforms: six places a Claude workload can run, and what actually decides between them

Q17 — Answer: A

- **Why A is correct:** A `thinking` block carries a signature that verifies it has not been edited, and the rule is that it must be passed back to the API unchanged on later turns. Any modification — even a faithful summary — breaks that signature and the request is rejected, and redacted thinking blocks are held to the same rule despite being encrypted and unreadable. The notary service has coupled its audit representation to its replay representation; the fix is to keep them separate, archiving the condensed text for auditors while replaying the original blocks verbatim.
- **Why not B:** Omission-by-default applies to the newest models; the service is on Haiku 4.5 with extended thinking enabled, which is what returns explicit `thinking` blocks in the first place.
- **Why not C:** Replaying thinking blocks is expected and supported — it is editing them that fails, not including them.
- **Why not D:** Nothing about extended thinking prevents resumption, and switching model tiers to dodge a signature rule discards the explicit reasoning the audit requirement depends on.
- **Difficulty:** Hard
- **Domain:** Tools and MCP
- **Tag:** `tools.implementation/thinking-block-signature`
- **Revise:** `3_claude_code_tools_mcp.md` → Message block structure and the pairing rule that isn't fixable by prompting

Q18 — Answer: B

- **Why B is correct:** Subagents improve execution along three axes: context isolation (a subagent's intermediate tool calls and reasoning never pollute the parent's context), parallelization, and specialization (a narrow, deep system prompt plus a tool set scoped down to reduce the blast radius of a mistake). That is what the pattern buys structurally, and the corresponding cost is what the manager is pointing at: the supervisor gets the summary rather than the intermediate work, and every subagent runs in its own context window spending its own tokens — the fan-out bill that makes this a hiring decision rather than a free upgrade.
- **Why not A:** Subagents do not share the parent's window; a fresh, unshared window is the defining property of the pattern.
- **Why not C:** Subagents inherit no conversation, files, or state from the parent, so the intermediate record is not preserved on the parent side.
- **Why not D:** Step-level observability through standard tooling is a *workflow* property; delegating to subagents moves you further toward transcript-level observability, not back from it.
- **Difficulty:** Medium
- **Domain:** Agents and Workflows
- **Tag:** `agents.architecture/subagent-context-isolation`
- **Revise:** `1_agents_and_workflows.md` → The role of subagents in improving task execution

Q19 — Answer: C

- **Why C is correct:** There are two distinct Bedrock surfaces. The legacy one uses AWS identity and billing with the `InvokeModel` / `Converse` APIs and ARN-versioned model identifiers, and pinning there is done through the ARN's version — a legitimate configuration for an existing integration that hasn't been

migrated to the Messages API surface. Claude in Amazon Bedrock is the newer surface, serving the Messages API at `/anthropic/v1/messages` with full model IDs under an `anthropic.` prefix. The new hire has read the description of the surface the brewery would migrate *to* and applied it to the one they are on.

- **Why not A:** An ARN-versioned identifier is exactly how pinning works on the legacy surface, so the deployment is pinned.
- **Why not B:** This swaps the two surfaces' API shapes; `/anthropic/v1/messages` is the Bedrock Messages API path.
- **Why not D:** They are genuinely different surfaces with different identity models, APIs, and pinning mechanics — not two spellings of one identifier.
- **Difficulty:** Hard
- **Domain:** Applications and Integration
- **Tag:** `apps.config/legacy-bedrock-arn-pinning`
- **Revise:** `2_applications_and_integration.md` → Deployment platforms: six places a Claude workload can run, and what actually decides between them

Q20 — Answer: D

- **Why D is correct:** The Message Batches API is a different submission model, not a faster one: you submit a large set of requests in one call, get an identifier back, and poll for completion, with jobs taking up to 24 hours and running at a lower per-token cost in exchange for that latency. It is built for offline pipelines, eval runs, and bulk jobs where nobody is waiting — an eval re-run is the textbook case. Twenty minutes without results is the tradeoff working as designed, not a degradation.
- **Why not A:** Turnaround is a property of the batch submission model, not of the longest request in the set.
- **Why not B:** Slicing the work into smaller submissions is the "chunking is not batching" mistake in reverse — smaller batches don't buy back the 24-hour window.
- **Why not C:** Eval runs are named as a batch use case precisely because they are latency-tolerant; the discount applies exactly here.
- **Difficulty:** Easy
- **Domain:** Model Selection and Optimization
- **Tag:** `mso.cost/batch-window-and-rate`
- **Revise:** `4_model_selection_prompting_context.md` → Technical Fundamentals

Q21 — Answer: A

- **Why A is correct:** Hooks and least-privilege roles share one dependency: they only cover the path or endpoint someone explicitly wrote a check for, which is why a hook watching `write_file` doesn't touch a network call to an unreviewed address. OS-level sandboxing isolates the agent at the process level instead of the rule level — filesystem isolation confines it to its working directory regardless of what any hook permits, and network isolation limits outbound connections to a named endpoint set regardless of what the identity role allows. Because the OS enforces it rather than application logic, it holds when a rule is missing or misconfigured, which is what closes the gap between "we have hooks" and "we have a defensible boundary."
- **Why not B:** Adding one more explicit check leaves the same structural weakness — the next uncovered endpoint or tool is another gap of exactly this kind.
- **Why not C:** Least privilege does bound the blast radius, but scoping it by "was there untrusted content in context" is application logic making the same rule-level bet that just failed.
- **Why not D:** Delimiting content as data is a soft boundary the injected text can argue around; the reliable boundary is in what the agent is allowed to *do* as a consequence of reading it.
- **Difficulty:** Hard
- **Domain:** Security and Safety

- **Tag:** `sec.guardrails/os-level-sandboxing`
- **Revise:** `5_eval_debugging_security.md` → OS-level sandboxing — the residual control

Q22 — Answer: B

- **Why B is correct:** On partner platforms — Vertex AI and Bedrock alike — retirement dates differ from Anthropic's own schedule, so availability has to be tracked against the platform the workload actually runs on rather than against Anthropic's announcements. The pin protects the deployment from a silent behavior shift; it does nothing about the model eventually going away on that platform's timetable. When the move does come, it is a version promotion like any other: gate it on the eval suite against the pinned baseline and retain the prior pin.
- **Why not A:** Pinning fixes *which* snapshot you get, not *how long* it stays available — a pinned ID is exactly what a retirement date applies to.
- **Why not C:** The rule is that the schedules differ, not that partners are always later; assuming a direction is the same mistake in a friendlier disguise.
- **Why not D:** Vertex does name the model in the endpoint URL, but that is an API-shape detail and does not hand version lifecycle management to the platform.
- **Difficulty:** Medium
- **Domain:** Applications and Integration
- **Tag:** `apps.config/partner-retirement-schedules`
- **Revise:** `2_applications_and_integration.md` → Deployment platforms: six places a Claude workload can run, and what actually decides between them

Q23 — Answer: C

- **Why C is correct:** The built-in `Explore` and `Plan` subagents are optimized for speed and skip `CLAUDE.md` and `git status` entirely, while `general-purpose` loads both. That single property explains both symptoms together: the `legacy_pricing/` restriction never reached the delegated work because `CLAUDE.md` wasn't loaded, and the plan treats reverted files as current because `git status` wasn't read. When a project rule silently doesn't apply to a delegated task, this is the usual reason.
- **Why not A:** Dilution is a real `CLAUDE.md` failure mode, but it degrades a rule's weight in a file that *did* load — here the file never loaded at all, and dilution wouldn't explain the stale `git` state.
- **Why not B:** Rules-file scoping comes from a `paths` glob in frontmatter, not from directory placement, and `CLAUDE.md` loads for the whole project rather than one directory.
- **Why not D:** `plan` mode holding the session in the explore phase is a permissions behavior, not the reason a delegated subagent never saw the rule, and compaction doesn't explain a `git status` that was never read.
- **Difficulty:** Hard
- **Domain:** Claude Code
- **Tag:** `cc.operation/builtin-subagents-skip-claude-md`
- **Revise:** `3_claude_code_tools_mcp.md` → `CLAUDE.md`, rules files, hooks, and subagents — four mechanisms, four different jobs

Q24 — Answer: D

- **Why D is correct:** The diagnostic table maps each symptom to the one structural technique that fixes it, and "task is right but the structure is invented — Claude did the task but in a shape you never specified" is the row for missing few-shot examples: a description alone can't pin down an exact structure, while an example shows it. The stem also names the anti-pattern the archive has been following — a prompt that keeps getting longer with every iteration and still fails is the signal that diagnosis is being skipped in favor of padding text. XML tags around each example keep Claude from reading the examples as part of the live instruction.

- **Why not A:** An output constraint is the fix when the form was never specified at all; here the prose specifies line order, labels, and units, and the output still lands in the wrong structure.
- **Why not B:** The system-prompt row is for content problems — scope drift, tone shifts, a broader question answered — and the assessments are accurate.
- **Why not C:** A missing variant constraint shows up as clean behavior on tested inputs breaking on an edge case, not as an invented structure across a third of ordinary reels.
- **Difficulty:** Medium
- **Domain:** Prompt and Context Engineering
- **Tag:** `prompt.prompting/few-shot-for-invented-structure`
- **Revise:** `4_model_selection_prompting_context.md` → Prompt Engineering

Q25 — Answer: C

- **Why C is correct:** An alias is a bare tier name like `sonnet` or `opus`, which resolves to a recommended version that updates over time and can differ by platform. A full model ID is a fixed snapshot — and starting with the Claude 4.6 generation, that full ID is dateless while still being pinned, so `claude-sonnet-5` is a pin rather than a moving target. Only earlier models need the ID plus a date suffix, as in `claude-haiku-4-5-20251001`. The reviewer has applied the older generation's format rule to a current-generation ID.
- **Why not A:** The date suffix is the pre-4.6 convention, not a universal marker of a pinned ID.
- **Why not B:** The `anthropic.` prefix is the Bedrock model-ID format; it identifies a platform's ID convention and has nothing to do with pinning on the first-party API.
- **Why not D:** Pinning is expressed in the model string your code sends; the versioning header on a partner platform (Vertex's `anthropic_version` body field) is a separate, unrelated requirement.
- **Difficulty:** Easy
- **Domain:** Applications and Integration
- **Tag:** `apps.config/dateless-pinned-model-ids`
- **Revise:** `2_applications_and_integration.md` → Configuration Management

Q26 — Answer: A

- **Why A is correct:** The gating question is whether waiting and retrying the identical request could plausibly work: 429 and 529 are retrievable, while 400 fails identically every time. The notes are equally explicit about the second half of the fix — check what the SDK already retries before writing your own loop, because the client libraries retry transient failures with progressive delay, and stacking an application loop on top multiplies attempts against the same rate limit rather than capping them. `retry-after` on a 429/529 is the service saying when capacity returns, and it beats guessing with backoff.
- **Why not B:** Backoff changes the timing of a retry, not whether the request can ever succeed — a 400 retried politely is still a 400, which is exactly what hid the malformed tool block.
- **Why not C:** The SDK retries transient failures only; authentication, permission, and bad-request failures are terminal, and the application still owns its own fallbacks.
- **Why not D:** The documented default runs the other way — when unsure, treat a failure as terminal, because a misclassified terminal failure fails loudly and gets fixed while a misclassified retrievable one hammers the service.
- **Difficulty:** Medium
- **Domain:** Eval, Testing, and Debugging
- **Tag:** `eval.debugging/retry-classification`
- **Revise:** `5_eval_debugging_security.md` → Failure handling: is it retrievable, or terminal?

Q27 — Answer: B

- **Why B is correct:** Pinning protects production from an unannounced upstream shift, but it does nothing about a shift you introduce yourself by moving the pin. The guidance is to retain the prior pinned version specifically so a regression is a rollback rather than a hotfix, and the documented incident is a team that had no prior pin to return to and had to patch a downstream parser instead. A traffic slice compared against a baseline is the promotion gate, not proof that every behavior on full traffic matches.
- **Why not A:** A pin fixes the snapshot the request resolves to; it says nothing about whether the new snapshot behaves like the old one, which is the regression in question.
- **Why not C:** An alias resolves to a recommended version that moves over time and can differ by platform, so it is a moving target rather than a rollback target.
- **Why not D:** The baseline score is an eval artifact the team owns and records; deleting a model ID constant does not delete the recorded baseline.
- **Difficulty:** Easy
- **Domain:** Applications and Integration
- **Tag:** `apps.config/prior-pin-rollback`
- **Revise:** `2_applications_and_integration.md` → Version pinning: an alias is a moving target, a full model ID is a fixed snapshot

Q28 — Answer: C

- **Why C is correct:** Anthropic's own multi-agent research system reported a substantial quality gain at roughly 15x the token cost of a normal single-agent interaction, because a lead plus several workers plus a synthesis pass each run in their own context window with their own input and output tokens. That multiplier is only earned when the work genuinely decomposes into independent parts explored in parallel; on tightly-coupled work where each step depends on the last — coding is the named counterexample — subagents mostly wait on each other, so you pay the fan-out without the parallel benefit. The stem is the documented incident pattern almost line for line, and the documented resolution is moving the task back to a single agent with the same context.
- **Why not A:** More workers on a chain whose steps depend on each other adds more waiting and more contexts to pay for; the constraint is the task's structure, not the number of slices.
- **Why not B:** Context isolation is a stated benefit of subagents, not a defect to undo, and rehydrating every worker with the parent's full history would raise the token bill rather than lower it.
- **Why not D:** The multiplier is a consequence of running several contexts, and it is worth paying on genuinely parallel-decomposable work; `effort` tunes reasoning depth and does not change whether the fan-out buys anything.
- **Difficulty:** Hard
- **Domain:** Agents and Workflows
- **Tag:** `agents.architecture/fan-out-multiplier`
- **Revise:** `1_agents_and_workflows.md` → The concrete cost of orchestrator-workers: treat it as a hiring decision

Q29 — Answer: D

- **Why D is correct:** Fable 5 is positioned as the most capable widely-released model and the tier for long-running agents, with a 1M-token context window and adaptive thinking that is **always on**. Adaptive thinking means the model itself decides when and how much to think, tuned through the `effort` setting — there is no enable flag to send and no token budget to size.
- **Why not A:** 200K is the Haiku 4.5 window, and `thinking.type: "enabled"` is the extended-thinking control that Haiku 4.5 supports rather than a Fable 5 one.

- **Why not B:** Adaptive thinking is not an opt-in on this tier; treating it as a flag to set is the standard confusion between adaptive and extended thinking.
- **Why not C:** The reasoning mode is adaptive rather than extended, and `budget_tokens` is deprecated in favor of `effort` and returns a 400 error on the newest generations.
- **Difficulty:** Easy
- **Domain:** Model Selection and Optimization
- **Tag:** `mso.model-choice/fable-5-capabilities`
- **Revise:** `4_model_selection_prompting_context.md` → Model Selection and Tradeoffs

Q30 — Answer: A

- **Why A is correct:** The Messages API is an HTTP REST API returning JSON, and an official SDK is a thin convenience layer over it that handles auth, request construction, retries, and response parsing. Because the transport is unchanged, every Claude application inherits ordinary REST concerns — idempotency, status-code handling along the retrievable/terminal split, and request/response schema validation — and those live in the application, which is why the review has to cover them.
- **Why not B:** The SDK removes boilerplate; it does not decide whether re-submitting an escrow summary is safe or whether a response's shape still matches what downstream code reads.
- **Why not C:** Typed response objects describe the shape the client expects, not a guarantee about content, and the client retries transient failures only — terminal statuses still reach the caller.
- **Why not D:** SDK and raw REST reach the same endpoints and the same models; the SDK surfaces API errors rather than replacing HTTP semantics with purely local ones.
- **Difficulty:** Medium
- **Domain:** Applications and Integration
- **Tag:** `apps.swe/rest-json-fundamentals`
- **Revise:** `2_applications_and_integration.md` → Software Engineering Foundations

Q31 — Answer: B

- **Why B is correct:** MCP adds a finer permission grain than server-level access: a rule can target one tool on one server with the `mcp__server__tool` form, so `mcp__mes__set_furnace_setpoint` is addressable on its own. The rule that resolves the conflict is that a deny on one tool overrides an allow on the whole server, and a deny always wins over an allow regardless of permission mode — which is precisely how the read-only tools stay auto-approved while the setpoint call still surfaces for a human.
- **Why not A:** Enumerating every read-only tool is unnecessary work built on the false premise that rules are server-wide only; per-tool targeting is exactly what `mcp__server__tool` provides.
- **Why not C:** `dontAsk` auto-denies anything not on the allow list with no confirmation queue — it is built for locked-down CI, and it classifies by allow list rather than by whether a tool mutates state.
- **Why not D:** Deny precedence is not scope-dependent; a deny outranks an allow wherever it sits, and an enterprise-managed deny is the most durable form precisely because it cannot be overridden.
- **Difficulty:** Medium
- **Domain:** Tools and MCP
- **Tag:** `tools.mcp/per-tool-permission-granularity`
- **Revise:** `3_claude_code_tools_mcp.md` → Tool Implementation

Q32 — Answer: C

- **Why C is correct:** An Agent SDK session follows a fixed message sequence: a `SystemMessage` with `subtype: "init"` carrying session metadata, then a cycle of `AssistantMessage` (text and/or tool-call requests) and `UserMessage` tool results, repeating until Claude answers with no tool calls, then a final

`AssistantMessage` and a `ResultMessage` carrying final text, token usage, cost, and session ID. Everything the auditors want is in the first and last message of that sequence, and the `ResultMessage` subtype is what names the ending — `success`, or `error_max_turns` / `error_max_budget_usd` when a cap was hit.

- **Why not A:** The ordering is inverted, and hitting `max_turns` or `max_budget_usd` returns a `ResultMessage` with an error subtype rather than raising mid-loop.
- **Why not B:** Tool results come back as a `UserMessage`, and there is one `ResultMessage` at the end of the session rather than one per turn.
- **Why not D:** `ClaudeAgentOptions` is the input configuration — limits, tools, effort — not a place run metadata is written back to.
- **Difficulty:** Medium
- **Domain:** Agents and Workflows
- **Tag:** `agents.construction/sdk-message-sequence`
- **Revise:** `1_agents_and_workflows.md` → The Claude Agent SDK and the agent loop

Q33 — Answer: D

- **Why D is correct:** Large-scale refactoring for a Claude application is named directly as migrating a workflow architecture to an agent architecture (or the reverse) as requirements shift from predictable to open-ended, which is exactly what unpredictable voice notes and varying telemetry have done here. The decision rule is whether the exact steps can be mapped in advance; a workflow where an agent is needed is documented as producing a system that breaks the moment user input deviates from the predetermined path, which is the eleven branches. The tradeoff is stated too: flexibility bought at the price of predictability, with actions bounded by the registered toolset and observability moving to the transcript level.
- **Why not A:** Small-scale refactoring is tightening a tool description or splitting an over-broad tool in two; replacing the orchestration layer is not that class of change.
- **Why not B:** A change of architecture is exactly what the eval suite exists to validate against its pinned baseline — the suite is what makes the migration reviewable rather than being invalidated by it.
- **Why not C:** Enumerating more branches is what has already failed; when inputs cannot be enumerated in advance, more branches is the wrong direction, and a human gate before a dispatch action is available inside an agent architecture too.
- **Difficulty:** Hard
- **Domain:** Applications and Integration
- **Tag:** `apps.swe/large-scale-refactor`
- **Revise:** `2_applications_and_integration.md` → Software Engineering Foundations

Q34 — Answer: A

- **Why A is correct:** Routing applies workflow routing to model choice: a cheap classification call reads a request-level signal — task type, input length, a difficulty label — and sends the bulk of traffic to a default tier while routing only the requests that need it to a larger model, so you pay for extra capability only where an eval shows it earned. The stated condition is the one the stem satisfies: the classification call only pays for itself when the traffic mix is genuinely heterogeneous, and you skip the router and pin a single model when traffic is uniform in shape.
- **Why not B:** Caching reuses a processed prefix and is a real cost lever, but it does not stop high-volume lookups being served by an unnecessarily capable tier, which is where the spend actually is.
- **Why not C:** Escalate-on-failure pays the cheap tier plus the capable tier on the hard slice and needs a validator that can recognize a bad analysis, which is the harder problem than classifying the request.
- **Why not D:** This pushes the routing decision onto callers who lack the signal, and the classifier is specifically described as a cheap call that pays for itself on heterogeneous traffic.

- **Difficulty:** Medium
- **Domain:** Model Selection and Optimization
- **Tag:** `mso.model-choice/classifier-routing`
- **Revise:** `4_model_selection_prompting_context.md` → Model Selection and Tradeoffs

Q35 — Answer: B

- **Why B is correct:** A prompt-level instruction about output is a request that holds on tested inputs and slips on the ones you didn't test. Structured outputs move the guarantee into the API through constrained decoding, and the variant that applies to tool arguments is strict tool use — `strict: true` on the tool definition constrains the arguments Claude passes to your tools against that tool's input schema before your code runs. The notes name this case exactly: use it in agentic loops where a malformed tool argument would crash the function or trigger the wrong action.
- **Why not A:** JSON outputs constrain the model's **final response**, which is a different surface from the arguments it hands to a tool mid-loop.
- **Why not C:** `required` describes which fields are meaningful, not an enforcement mechanism — and over-marking fields required pushes Claude to fabricate values it has no basis for.
- **Why not D:** A hook blocks a malformed call after it has been generated, short-circuiting the turn; strict tool use prevents the invalid argument from being emitted in the first place.
- **Difficulty:** Medium
- **Domain:** Prompt and Context Engineering
- **Tag:** `prompt.output/strict-tool-use`
- **Revise:** `4_model_selection_prompting_context.md` → Output Handling

Q36 — Answer: C

- **Why C is correct:** Tool input/output schemas and structured-output JSON schemas are part of the application's API contract, and the guidance is to treat a schema change with the same discipline as a public API's breaking-change policy — because it can silently break every caller relying on the old shape, with no compiler to catch it. That is reinforced on the process side: code review for a Claude application should explicitly cover prompt and tool-schema diffs, since a schema change is a breaking-change risk for every caller of that tool. A tool re-exposed through MCP is indistinguishable from a manually registered one, so the portal inherits the break too.
- **Why not A:** Selection quality is driven by the description and is a real concern, but it is not the only one — the schema is also the shape every caller serializes against.
- **Why not B:** An eval grades model behavior against expected outputs; it does not exercise three internal applications' serialization of the old field name.
- **Why not D:** Adding a required field is a classic breaking change for callers that never sent it, and over-marking fields required also pushes Claude to fabricate values it has no basis for.
- **Difficulty:** Medium
- **Domain:** Applications and Integration
- **Tag:** `apps.design/schema-as-api-contract`
- **Revise:** `2_applications_and_integration.md` → Schema design

Q37 — Answer: D

- **Why D is correct:** Least privilege is what bounds the blast radius once an injection or a bad decision gets through every earlier layer — but that bound only holds if the role itself is fixed. The notes call this out as a subtle but critical detail: anything that can modify the agent's own auth or role configuration can

effectively act with that identity, so protecting that configuration matters as much as protecting the secret, and editing the role is a privileged action belonging behind the same protection. A role file inside the agent's writable scope makes every deny in it advisory.

- **Why not A:** The role in the stem is explicit about both its allow paths and its denies; the defect is not the shape of the policy but that the policy is editable by the subject it governs.
- **Why not B:** Hooks running in the application process rather than the model's context is precisely why they are a deterministic, code-level guarantee, not a weakness.
- **Why not C:** Network isolation is a real residual control, but it addresses outbound connections rather than a write to a role file that already sits in the agent's permitted directory.
- **Difficulty:** Hard
- **Domain:** Security and Safety
- **Tag:** `sec.appsec/auth-config-as-privileged-target`
- **Revise:** `5_eval_debugging_security.md` → Least privilege: the control that holds even when every other layer fails

Q38 — Answer: A

- **Why A is correct:** Managed Agents fits the operational profile in the stem — long-running execution and a managed sandbox are exactly what it is for — but its sessions are stateful and stored server-side, and it is currently not eligible for Zero Data Retention or a HIPAA BAA. That is stated as the governing constraint that overrides convenience: a workload carrying PHI or under a ZDR requirement rules Managed Agents out regardless of operational fit, and routes to the Agent SDK or a raw loop on a covered configuration instead.
- **Why not B:** ZDR eligibility varies by model and platform and is not guaranteed even under an existing ZDR agreement; a beta header does not extend coverage to an ineligible surface.
- **Why not C:** Eligibility is a property of the model *and* the platform/surface together, which is why each candidate configuration has to be confirmed rather than assumed from the model alone.
- **Why not D:** Moving from the Agent SDK to Managed Agents is documented as a re-expression step — code and filesystem configuration become a versioned API resource — not a direct export, and it would not change the eligibility position anyway.
- **Difficulty:** Hard
- **Domain:** Agents and Workflows
- **Tag:** `agents.construction/managed-agents-phi-zdr`
- **Revise:** `1_agents_and_workflows.md` → Managed agent deployment models: three wiring paths, ordered by how much infrastructure you hand off

Q39 — Answer: B

- **Why B is correct:** The documented levels run `low` (minimal reasoning, fast — file lookups and listing), `medium` (balanced), `high` (thorough analysis), `xhigh` (extended reasoning depth for complex coding and agentic tasks) and `max` (maximum depth for multi-step problems needing deep analysis). The survey-file listing is the canonical `low` case, while a multi-step reconciliation across several constraints is exactly what `xhigh` and `max` exist for. Reasoning earns its cost on hard multi-step problems and is wasted on lookups and classification.
- **Why not A:** `xhigh` and `max` are effort levels of the same setting, described from the Agent SDK but applying at the raw API level too, not Claude Code-only options.
- **Why not C:** Depth costs tokens and latency on every call, which is why it is wasted on a file listing; a single agent can vary effort per call.
- **Why not D:** `budget_tokens` is deprecated in favor of `effort` and returns a 400 error on the newest model generations.

- **Difficulty:** Easy
- **Domain:** Model Selection and Optimization
- **Tag:** `mso.technical/effort-levels`
- **Revise:** `4_model_selection_prompting_context.md` → Model Selection and Tradeoffs

Q40 — Answer: C

- **Why C is correct:** `settings.json` follows a five-scope precedence — managed > CLI > local > project > user — with one documented exception: permissions accumulate rather than override. So the developer's local allow rule is added to the project's rules rather than replacing them, and the user-scope file is not discarded just because a project file exists. The enterprise-managed scope remains the most durable control available, because it cannot be overridden by a user or project file and a deny rule always wins over an allow, even under a bypass mode.
- **Why not A:** The ordering is inverted — managed settings are the highest-precedence scope, specifically so org-wide security controls cannot be overridden locally.
- **Why not B:** The ordering is right but the "every key uniformly" claim misses the permissions-accumulate exception, which is the whole point of the question.
- **Why not D:** Managed scope is not a set of defaults; it cannot be overridden by users or project files, and a project allow cannot widen a managed deny.
- **Difficulty:** Hard
- **Domain:** Applications and Integration
- **Tag:** `apps.config/settings-scope-precedence`
- **Revise:** `2_applications_and_integration.md` → Configuration Management

Q41 — Answer: D

- **Why D is correct:** The API MCP connector's `mcp_toolset` object carries a `default_config` applied to every tool on a server, overridable per-tool through `configs` keyed by tool name, and the two settings that matter for context cost are `defer_loading` (delay a tool's definition until the model actually needs it) and `enabled` (register a whole server but expose only a subset). It requires the `mcp-client-2025-11-20` beta header, without which the configuration doesn't apply. The constraint that decides this scenario is that the connector supports remote HTTP servers only — a local stdio server needs Claude Desktop or Claude Code as its client and cannot be connected directly through the API.
- **Why not A:** Transport is not abstracted away; stdio is precisely what the connector cannot reach.
- **Why not B:** `enabled` is real, but `defer_loading` is a documented `mcp_toolset` setting rather than a permission behavior, and the stdio problem is still unaddressed.
- **Why not C:** Per-tool overrides are explicitly supported through `configs` keyed by tool name.
- **Difficulty:** Hard
- **Domain:** Tools and MCP
- **Tag:** `tools.mcp/connector-toolset-config`
- **Revise:** `3_claude_code_tools_mcp.md` → MCP Server Development

Q42 — Answer: A

- **Why A is correct:** Claude Code works explore → plan → code, and `plan` mode is the permission mode that holds the session in the explore phase: reads are auto-approved so it can research and propose, and every edit and command is blocked until the plan is approved and released. That is the same shape as the human gate placed after a planning step, which addresses the risk that an incorrect plan produces the wrong outcome even when every step executes correctly.
- **Why not B:** `default` auto-approves reads and prompts on each edit or command individually, so the session moves into the code phase as soon as one prompt is approved — nothing holds it at the proposal.

- **Why not C:** `acceptEdits` auto-approves edits inside the working directory, which is the opposite of a guarantee that nothing is written before review.
- **Why not D:** `dontAsk` auto-denies anything off the allow list with no confirmation queue — it is built for locked-down CI, and it gives the human no way to approve the plan and proceed.
- **Difficulty:** Easy
- **Domain:** Claude Code
- **Tag:** `cc.operation/plan-mode`
- **Revise:** `3_claude_code_tools_mcp.md` → Permission modes — a risk decision, not a speed decision

Q43 — Answer: B

- **Why B is correct:** `PreCompact` fires before context compaction and its documented use is archiving the full transcript before it is summarized away, which is exactly the compliance requirement. `SubagentStart` and `SubagentStop` fire when a subagent spawns and completes, and their documented use is tracking and aggregating parallel results. The cost answer matters too: hooks are callbacks that run in your application process, not inside the model's context window, so they don't consume tokens.
- **Why not A:** A subagent dispatch is not surfaced as an ordinary tool call for this purpose, and `PreToolUse` fires per tool rather than before compaction — nor does hook output land in the model's context.
- **Why not C:** Compaction happens as the window approaches its limit, not at end of turn, and `SubagentStart` is a documented event.
- **Why not D:** `SessionEnd` runs at teardown, after compaction has already discarded the detail, and `Notification` fires on a permission request or after 60 seconds idle.
- **Difficulty:** Medium
- **Domain:** Agents and Workflows
- **Tag:** `agents.construction/hook-events`
- **Revise:** `1_agents_and_workflows.md` → Hooks for deterministic actions

Q44 — Answer: C

- **Why C is correct:** The instruction hierarchy differs per interface. Claude Desktop and `claude.ai` persist instructions through custom instructions and Projects, which are user- or workspace-scoped rather than file-based — there is no file for the compliance officer to review or version. On the API and SDKs, persistent instructions come from the `system` parameter set per request, fully under the application's control with no implicit persistence. The design mistake the notes name is assuming behavior tuned against one interface transfers unchanged to the raw API; and prompts are production configuration, tracked with version control, changelogs, and rollback.
- **Why not A:** `CLAUDE.md` is the Claude Code mechanism; a service calling the API has no `CLAUDE.md` and every persistent instruction must be re-sent in `system`.
- **Why not B:** There is no cross-surface persistence to inherit — Project instructions are scoped to that workspace, not to every deployment the organization runs.
- **Why not D:** Project instructions are not an artifact the API can reference; the reviewable, versioned form is the prompt held in the application's own version control.
- **Difficulty:** Easy
- **Domain:** Applications and Integration
- **Tag:** `apps.design/instruction-persistence`
- **Revise:** `2_applications_and_integration.md` → How Claude interprets instructions across interfaces

Q45 — Answer: D

- **Why D is correct:** This is the documented context-cap incident. A deliberate cap well under the model's ceiling was validated against dev fixtures averaging small tool results; production inputs carried far more supporting material, and the budget filled several turns before the task finished. The failure *looked* like degraded tool selection, but the cause was accumulated, never-pruned tool output crowding out the system prompt and early instructions that told the agent which tool to use next. The stated tell is precisely the stem's: if tool selection degrades after a consistent number of turns, check whether the window is filling before debugging the schema — and measure a tool result against the largest realistic production input, pruning or compacting proactively.
- **Why not A:** Overlapping descriptions cause wrong-tool calls from the very first ambiguous request, not reliably from turn 8 onward, so the turn-count regularity points elsewhere.
- **Why not B:** `model_context_window_exceeded` is a partial response when generation hits the model's own window mid-generation; a self-imposed budget cap is an application-level control and this failure is degradation across turns, not a truncated single response.
- **Why not C:** A swallowed tool error produces a confident wrong answer built on missing data, which is a real failure mode but not one that switches on at a fixed turn count.
- **Difficulty:** Hard
- **Domain:** Prompt and Context Engineering
- **Tag:** `prompt.context/context-overflow-misdiagnosis`
- **Revise:** `4_model_selection_prompting_context.md` → Context Engineering

Q46 — Answer: A

- **Why A is correct:** Three routes are named as authorized: Claude for Government (FedRAMP High via Palantir Federal Cloud), Claude via Amazon Bedrock GovCloud (FedRAMP High, DoD IL4/5), and Claude via Vertex AI Assured Workloads. Claude Enterprise on the AWS Marketplace is called out explicitly as *not* FedRAMP authorized, so being on AWS decides nothing on its own. The constraint also covers the non-production path: what a FedRAMP requirement rules out is any code path hitting an endpoint outside the authorized cloud environment at the required impact level, including dev and test paths that hit the commercial endpoint while production hits the authorized one.
- **Why not B:** Authorization attaches to the specific access route and environment, not to the vendor or the billing channel — standard Bedrock and Bedrock GovCloud are not interchangeable here.
- **Why not C:** It has production and staging backwards: the Marketplace route is the unauthorized one, and staging matters as an additional exposure rather than the only problem.
- **Why not D:** Two of the three authorized routes are cloud-mediated (Bedrock GovCloud and Vertex AI Assured Workloads); the direct API is not the sole federal path.
- **Difficulty:** Medium
- **Domain:** Security and Safety
- **Tag:** `sec.appsec/fedramp-authorized-routes`
- **Revise:** `5_eval_debugging_security.md` → Regulated data constraints: what each rules out in code, before a single prompt is written

Q47 — Answer: B

- **Why B is correct:** Latency is one of three platform-comparison dimensions that require actual measurement rather than assumption, and the stated method is to measure from the customer's actual region against their actual payload. A measurement taken from your own laptop hides the round-trip penalty that appears once the workload runs where the customer actually is — which is the whole error in the stem, compounded by a test prompt that isn't a real request. Latency is also an infrastructure requirement in its own right: how fast, measured where the user actually is.

- **Why not A:** Per-token rates being broadly aligned is the *cost* dimension; latency is separately measurable and genuinely differs, since an in-region platform shortens round-trip time.
- **Why not C:** Streaming changes when output starts appearing, not the network distance between the caller and the endpoint, so it doesn't remove the location dependence.
- **Why not D:** Payload size affects both, which is why the method specifies the customer's *actual* payload rather than a convenient short one.
- **Difficulty:** Easy
- **Domain:** Applications and Integration
- **Tag:** `apps.requirements/latency-measurement`
- **Revise:** `2_applications_and_integration.md` → Comparing platforms so the choice survives a procurement/security review

Q48 — Answer: C

- **Why C is correct:** The two mechanisms are described distinctly. Automatic caching is one top-level `cache_control` field: the system applies the breakpoint to the last cacheable block and slides it forward as the conversation grows — which is what the first engineer wants. Explicit breakpoints place `cache_control` on individual content blocks, up to four per request, subject to a 20-block lookback window and a per-block minimum token threshold (about 1,024 tokens on current models). Because requests process in a fixed order — tools, then system prompt, then messages — a breakpoint placed after the tool definitions caches them while leaving the messages dynamic, which is what the second engineer wants.
- **Why not A:** The four-breakpoint limit belongs to explicit breakpoints, and automatic caching applies a single sliding breakpoint rather than marking every block.
- **Why not B:** An explicit breakpoint caches the prefix up to that point, and appending later messages does not invalidate it — only a change before the cache point does.
- **Why not D:** The 5-minute default and the 1-hour `tTL: "1h"` option are a separate TTL choice, not what distinguishes automatic from explicit breakpoints.
- **Difficulty:** Medium
- **Domain:** Model Selection and Optimization
- **Tag:** `mso.cost/cache-breakpoints`
- **Revise:** `4_model_selection_prompting_context.md` → Cost and Token Management

Q49 — Answer: A

- **Why A is correct:** Both approaches find a relevant slice of source material and generate from it; the difference is *when* the relevant-slice decision happens — at index-build time for classical RAG, which pre-builds an embedding index over chunked material and matches a query embedding at request time, or at query time for agentic search, which skips the index and has the model search and fetch live at the moment of need. MCP tool search and Claude.ai Projects' document surfacing are the named examples of the latter. Retrieval of either kind scales because request cost stays flat as the source material grows, and both are only as good as what they find, so badly organized material degrades results regardless of approach.
- **Why not B:** Agentic search fetches a relevant slice rather than loading the corpus, and flat request cost as material grows is a property of retrieval generally, not of RAG specifically.
- **Why not C:** Classical RAG matches embeddings rather than spending a model call per chunk at request time, and live search is the approach that naturally sees documents added moments ago.
- **Why not D:** They are two approaches to the same retrieval problem; agentic search is not a permission setting.
- **Difficulty:** Medium

- **Domain:** Tools and MCP
- **Tag:** `tools.mcp/rag-vs-agentic-search`
- **Revise:** `3_claude_code_tools_mcp.md` → MCP Server Development

Q50 — Answer: D

- **Why D is correct:** Of the three third-party frameworks the blueprint names, Strands Agents is AWS's open-source, model-driven agent framework, deeply integrated with Amazon Bedrock and the natural choice inside an AWS-centric stack. The framing the exam tests is that the Claude Agent SDK is a provider-native primitive — loop, tool execution, MCP integration, and prompt caching all built in and optimized for Claude — while Strands, LangGraph, and PydanticAI are independent cross-provider frameworks reached for when you need a pattern the native SDK doesn't offer, such as a specific cloud's native integration, not because the SDK is less capable at the core loop.
- **Why not A:** LangGraph's defining characteristic is graph-based orchestration with agent steps as explicit nodes for complex stateful workflows; it carries no particular AWS alignment.
- **Why not B:** PydanticAI is Python-first type safety and schema validation without heavy orchestration machinery, which is orthogonal to which cloud hosts the model.
- **Why not C:** The cross-provider frameworks are provider-agnostic by design — that is the point of them — so this inverts the actual distinction.
- **Difficulty:** Easy
- **Domain:** Agents and Workflows
- **Tag:** `agents.patterns/strands-bedrock`
- **Revise:** `1_agents_and_workflows.md` → Third-party agentic frameworks

Section D — Score and Analysis

1. Domain breakdown

Total your correct answers per domain using the question numbers listed, then fill in the last three columns.

Domain	Questions	Your score	%	Official weight	Weighted contribution
Applications and Integration — Q1, 4, 6, 9, 11, 14, 16, 19, 22, 25, 27, 30, 33, 36, 40, 44, 47	17	___ / 17	___%	33.1%	___
Model Selection and Optimization — Q3, 8, 13, 20, 29, 34, 39, 48	8	___ / 8	___%	16.8%	___
Agents and Workflows — Q2, 12, 18, 28, 32, 38, 43, 50	8	___ / 8	___%	14.7%	___
Prompt and Context Engineering — Q5, 15, 24, 35, 45	5	___ / 5	___%	11.0%	___
Tools and MCP — Q7, 17, 31, 41, 49	5	___ / 5	___%	10.6%	___
Security and Safety — Q10, 21, 37, 46	4	___ / 4	___%	8.1%	___
Claude Code — Q23, 42	2	___ / 2	___%	3.1%	___
Eval, Testing, and Debugging — Q26	1	___ / 1	___%	2.6%	___
Total	50	___ / 50	___%	100%	___

2. Weighted readiness

Readiness = $\sum$ (domain score % $\times$ official domain weight)

Worked example, for a sitting that scored 40/50 raw (80%) with the misses concentrated in one domain:

Domain	Score	%	Weight	Contribution
Applications and Integration	9/17	52.9%	33.1%	17.52
Model Selection and Optimization	8/8	100%	16.8%	16.80
Agents and Workflows	8/8	100%	14.7%	14.70
Prompt and Context Engineering	4/5	80.0%	11.0%	8.80
Tools and MCP	4/5	80.0%	10.6%	8.48
Security and Safety	4/4	100%	8.1%	8.10
Claude Code	2/2	100%	3.1%	3.10
Eval, Testing, and Debugging	1/1	100%	2.6%	2.60
Readiness				80.1%

Read that table carefully, because it is the trap this exam exists to expose. Raw 80%, weighted 80.1% — both comfortably in the "nearly ready" band — while the domain that is a third of the real exam is sitting at 52.9%, barely above a coin flip. The aggregate looks fine precisely because each mock's item counts mirror the official weights, so the weighted number tracks the raw one closely and neither aggregate can reveal a single collapsed domain. On this exam, the per-domain row is the signal and the total is nearly noise.

3. Readiness bands

Readiness	Reading	Next step
85%+	Ready to sit	Take Exam 4 as a timed final rehearsal
75–84%	Nearly ready	Two focused sessions on your two weakest domains, then Exam 4
65–74%	Real gaps	Re-run coaching sessions for every domain scoring under 70% before touching Exam 4 — you only get one final rehearsal, so don't spend it early
Below 65%	Not yet	Return to the source notes for the weakest domains; mocks measure, they don't teach

4. Weak-area capture

Do this before you look at anything else, while you still remember why you picked what you picked.

1. For **every** missed question, copy its **Tag** from Section C into the weak-areas table in `../progress_tracker.md`, with `M3 Q<n>` in the "Where it came from" column and the **Revise** pointer in the "Revise" column. A tag goes to `watching` on its first miss and `active` at two.
2. Write down *why* you missed it, not just that you did — "confused strict tool use with JSON outputs" is actionable; a tick in a box is not.
3. Record the raw score, percentage, weighted readiness, and the list of domains under 70% in the mock exam log, and fill the M3 column of the domain breakdown table.
4. **Any domain scoring under 70% is a weak area regardless of your overall score.** Its tags go straight to `active` without waiting for a second miss, and the corresponding coaching session gets re-run before Exam 4.
5. This exam is the reinforcement check, so run one extra comparison the others don't ask for: for each of the three weak areas you cleared before sitting it, find the questions here that carry those tags and check them specifically. A tag you missed again after reinforcement is not a revision problem — go back to the mechanism in the source file and work it until you can explain it aloud without notes, because re-reading has already failed once.